

MICROSAR Socket Adaptor

Technical Reference

Version 14.5.0

Authors	Alex Lunkenheimer, Marc Weber, Michael Dangelmaier, Philipp Christmann, Michael Seidenspinner, Jens Bauer, Jessica Elsäßer
Status	Released

Document Information

History

Author	Date	Version	Remarks
Alex Lunkenheimer	2008-11-20	1.0	Creation of document
Alex Lunkenheimer	2009-10-05	2.0	Tool based configuration
Marc Weber	2012-01-16	2.1	Call-back for Ethernet State Manager and minor changes
Marc Weber	2012-07-24	2.2	DoIP extensions; dynamic UDP port usage
Michael Dangelmaier	2012-10-04	2.3	DaVinci Configurator Pro support; AUTOSAR 4 support
Michael Dangelmaier	2013-07-10	2.4	Customer specific extensions
Michael Dangelmaier	2014-05-23	3.0	Updated to AUTOSAR 4.1
Michael Dangelmaier	2015-03-26	4.0	Updated to AUTOSAR 4.2
Michael Dangelmaier	2015-05-16	5.0	Optimized UDP retry behavior, Added BSD Socket API
Philipp Christmann	2015-11-16	6.0	Support of post-build loadable
Michael Dangelmaier	2016-03-23	6.1	Support of TLS client Trigger Transmit API with SduLength In/Out Description for shutdown mechanism
Michael Dangelmaier	2016-04-28	6.2	Release of BSD-Socket API
Michael Dangelmaier	2016-05-31	6.3	Extension of BSD Socket API to support SOME/IP-SD under Linux
Michael Dangelmaier	2016-11-11	7.0	MainFunction splitting Optimized TP transmission Trigger Transmit API for SoAd_IfTransmit Optimized buffer handling for PDU fan-out
Michael Dangelmaier	2017-01-23	7.1	Event Queues and Timeout Lists
Michael Dangelmaier	2017-02-22	7.2	Support Buffer Size up to 128kB
Michael Dangelmaier	2017-05-08	8.0	Updated component history
Michael Dangelmaier	2017-05-30	8.1	Updated service IDs
Michael Dangelmaier	2017-06-19	8.2	PDU reception verification Transmission on specific socket connection Forward socket connection on reception
Michael Dangelmaier	2017-08-01	8.3	Updated API description
Michael Dangelmaier	2017-08-28	8.4	Reworked critical section chapter
Michael Dangelmaier	2018-03-19	9.0	Reworked TP-API description
Michael Seidenspinner	2018-07-31	10.0	Support INTEGRITY
Michael Dangelmaier	2018-08-22	10.1	Support VLAN priorities for Linux
Michael Dangelmaier	2019-03-12	11.0	Updated version history
Michael Dangelmaier	2019-03-22	12.0	Added services to write/read/get events for DHCP options

Michael Dangelmaier	2019-06-25	12.1	Updated version history
Michael Dangelmaier	2019-07-24	13.0	Added DoIPInt to the supported modules
Michael Dangelmaier	2019-08-27	13.1	Added service to retrieve and reset measurement data
Jens Bauer	2020-02-13	14.0	Added description of additional critical sections
Michael Dangelmaier	2020-06-02	14.1	Updated version history
Michael Dangelmaier	2020-06-18	14.2	Updated version history
Jens Bauer	2020-07-14	14.3	Updated version history
Jessica Elsässer	2020-09-21	14.4	Support for disabling the broadcast/multicast reception for LINUX and QNX
Jessica Elsässer	2021-01-04	14.5	Support keeping online of socket connections after transmission on automatic socket connection setup

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_SocketAdaptor.pdf	4.2.2
[2]	AUTOSAR	AUTOSAR_SWS_SocketAdaptor.pdf	4.3.0
[3]	AUTOSAR	AUTOSAR_SWS_DevelopmentErrorTracer.pdf	4.1.2
[4]	AUTOSAR	AUTOSAR_SWS_DiagnosticEventManager.pdf	4.1.2
[5]	AUTOSAR	AUTOSAR_BasicSoftwareModules.pdf	V1.0.0
[6]	AUTOSAR	AUTOSAR_SWS_Tcplp.pdf	4.2.1

Scope of the Document

This technical reference describes the general use of the Socket Adaptor basis software. Please refer to your Release Notes to get a detailed description of the platform (host, compiler) your Vector Ethernet Bundle has been configured for.



Caution

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1	Component History	11
2	Introduction.....	13
2.1	Architecture Overview	15
3	Functional Description	17
3.1	Features	17
3.2	Deviations	18
3.2.1	Not supported standard conform features	18
3.2.2	Not supported optional features	18
3.2.3	Additions/ Extensions	18
3.2.4	Known Issues (low priority)	19
3.2.4.1	ESCAN00087305	19
3.2.5	Hints	19
3.2.5.1	CDD Contribution Type	19
3.2.5.2	API deviation.....	19
3.2.5.3	UDP socket ressources bound at startup	19
3.2.5.4	SoAd_SetUniqueRemoteAddress() disables alive supervision timeout	19
3.2.5.5	SoAd_CloseSoCon() if open/close counter is 0.....	19
3.3	Initialization	20
3.3.1	Configuration Variants 1, 2 (Pre-Compile and Link-Time).....	20
3.3.2	Configuration Variant 3 (Post-build).....	20
3.4	States	20
3.5	Main Functions	21
3.6	Error Handling.....	21
3.6.1	Development Error Reporting.....	21
3.6.2	Production Code Error Reporting	23
4	Integration.....	24
4.1	Scope of Delivery.....	24
4.2	Critical Sections	25
4.3	Main Functions	26
4.3.1	BSD Socket API	26
4.3.2	Preemption	26
5	API Description.....	27
5.1	Type Definitions	27
5.2	Services provided by Socket Adaptor.....	27
5.2.1	SoAd_InitMemory	27

5.2.2	SoAd_Init	28
5.2.3	SoAd_IfTransmit	28
5.2.4	SoAd_IfRoutingGroupTransmit	29
5.2.5	SoAd_IfSpecificRoutingGroupTransmit	29
5.2.6	SoAd_TpTransmit	30
5.2.7	SoAd_Shutdown	31
5.2.8	SoAd_TpCancelTransmit	31
5.2.9	SoAd_TpCancelReceive	32
5.2.10	SoAd_GetSoConId	32
5.2.11	SoAd_OpenSoCon	33
5.2.12	SoAd_CloseSoCon	33
5.2.13	SoAd_RequestIpAddrAssignment	34
5.2.14	SoAd_ReleaseIpAddrAssignment	35
5.2.15	SoAd_GetLocalAddr	35
5.2.16	SoAd_GetPhysAddr	36
5.2.17	SoAd_GetRemoteAddr	36
5.2.18	SoAd_GetRemoteAddrState	37
5.2.19	SoAd_GetRcvRemoteAddr	38
5.2.20	SoAd_EnableRouting	38
5.2.21	SoAd_EnableSpecificRouting	39
5.2.22	SoAd_DisableRouting	40
5.2.23	SoAd_DisableSpecificRouting	40
5.2.24	SoAd_SetRemoteAddr	41
5.2.25	SoAd_SetUniqueRemoteAddr	41
5.2.26	SoAd_ReadDhcpHostNameOption	42
5.2.27	SoAd_WriteDhcpHostNameOption	43
5.2.28	SoAd_ReadDhcpOption	43
5.2.29	SoAd_WriteDhcpOption	44
5.2.30	SoAd_GetVersionInfo	44
5.2.31	SoAd_GetAndResetMeasurementData	45
5.2.32	SoAd_MainFunctionRx	46
5.2.33	SoAd_MainFunctionState	46
5.2.34	SoAd_MainFunctionTx	47
5.2.35	SoAd_MainFunction	47
5.3	Services used by Socket Adaptor	48
5.4	Callback Functions	49
5.4.1	SoAd_RxIndication	49
5.4.2	SoAd_CopyTxData	50
5.4.3	SoAd_TxConfirmation	51
5.4.4	SoAd_LocalIpAddrAssignmentChg	51
5.4.5	SoAd_TcpAccepted	52

5.4.6	SoAd_TcpConnected	52
5.4.7	SoAd_TcpIpEvent	53
5.4.8	SoAd_DhcpEvent.....	53
5.5	Configurable Interfaces	54
5.5.1	<Up>_[SoAd][If]RxIndication	54
5.5.2	<Up>_[SoAd][If]TriggerTransmit.....	55
5.5.3	<Up>_[SoAd][If]TxConfirmation	55
5.5.4	<Up>_[SoAd][Tp]StartOfReception	56
5.5.5	<Up>_[SoAd][Tp]CopyRxData	56
5.5.6	<Up>_[SoAd][Tp]RxIndication	57
5.5.7	<Up>_[SoAd][Tp]CopyTxData.....	58
5.5.8	<Up>_[SoAd][Tp]TxConfirmation	58
5.5.9	<Up>_SoConModeChg.....	59
5.5.10	<Up>_LocalIpAddrAssignmentChg	59
5.5.11	<Up>_ShutdownFinished	60
5.5.12	<Up_TcpTlsSocketCreatedNotification>	60
5.5.13	<Up_VerifyRxPdu>	61
5.5.14	<Up_DhcpEvent>.....	62
6	Configuration	63
6.1	Configuration Variants.....	63
6.2	Configuration with DaVinci Configurator Pro	63
6.2.1	Socket Connection handling.....	63
6.2.1.1	Socket Connection Group	63
6.2.1.2	Socket Connection establishment	64
6.2.1.2.1	Manual.....	65
6.2.1.2.2	Automatic.....	66
6.2.2	Transmission path.....	66
6.2.3	Reception path.....	66
6.2.4	Vector specific feature.....	67
6.2.4.1	Best Match Algorithm extensions	67
6.2.4.2	UDP Immediate IF TxConfirmaion.....	68
6.2.4.3	SoAd_GetRcvRemoteAddr().....	68
6.2.4.4	Additional SoConModeChg notification	68
6.2.4.5	BSD Socket API	69
6.2.4.5.1	Enable/Disable BSD Socket API	69
6.2.4.5.2	Configure Socket API type	69
6.2.4.5.3	Configure header file for type "sockaddr_ll"	70
6.2.4.5.4	Configure local IP address	70
6.2.4.5.5	Request and release IP address	72

6.2.4.5.6	VLAN priority.....	72
6.2.4.5.7	Broadcast/Multicast Reception.....	73
6.2.4.6	Shutdown mechanism.....	73
6.2.4.7	TLS client.....	75
6.2.4.8	MainFunction splitting	76
6.2.4.9	Optimized TP transmission	77
6.2.4.10	Trigger Transmit API for SoAd_IfTransmit	78
6.2.4.11	Optimized buffer handling for PDU fan-out.....	79
6.2.4.12	Event Queues and Timeout Lists	80
6.2.4.12.1	Event Queues	80
6.2.4.12.2	Timeout Lists	81
6.2.4.13	PDU reception verification.....	81
6.2.4.14	“Transmission on specific socket connection” and “Forward socket connection on reception”	82
6.2.4.15	Get DHCP option events.....	83
6.2.4.16	Keep Socket Connection online after transmission	83
6.2.5	Complex Device Driver (CDD)	84
7	Glossary and Abbreviations	86
7.1	Glossary	86
7.2	Abbreviations	86
8	Contact.....	88

Illustrations

Figure 2-1	Functional Overview	14
Figure 2-2	AUTOSAR 4.2 Architecture Overview	15
Figure 2-3	Interfaces to adjacent modules of the Socket Adaptor	15
Figure 3-1	Module states	20
Figure 6-1	Socket Connection Group configuration.....	64
Figure 6-2	Socket Connection configuration	64
Figure 6-3	Socket Connection Setup parameter	65
Figure 6-4	Remote address “not set” parameter	65
Figure 6-5	Add PDU Route	66
Figure 6-6	Add PDU Route Destination	66
Figure 6-7	Add Socket Route.....	66
Figure 6-8	Add Socket Route Destination	67
Figure 6-9	Best Match Algorithm extensions	67
Figure 6-10	UDP Immediate IF TxConfirmation	68
Figure 6-11	SoAd_GetRcvRemoteAddr	68
Figure 6-12	Additional SoConModeChg notification.....	69
Figure 6-13	Socket API type configuration	70
Figure 6-14	Configure header file for type “sockaddr_ll”	70
Figure 6-15	Controller/Interface configuration	70
Figure 6-16	Address Type configuration	71
Figure 6-17	Assignment Method configuration.....	71
Figure 6-18	Assignment Trigger configuration.....	72
Figure 6-19	Frame Priority configuration.....	72
Figure 6-20	Set Egress Priority configuration.....	73
Figure 6-21	Set Broadcast Reception configuration	73
Figure 6-22	Enable upper layer specific <User>_ShutdownFinished()	74
Figure 6-23	Configure <User>_ShutdownFinished() for any module.....	74
Figure 6-24	Additional header file inclusion	74
Figure 6-25	Shutdown finished timeout.....	74
Figure 6-26	Create TLS client configuration in socket connection.....	75
Figure 6-27	SoAd Tls Tx and Rx buffer configuration.....	75
Figure 6-28	Tls callback container configuration	76
Figure 6-29	Tls callback container reference configuration	76
Figure 6-30	Optimized TP transmission sequence.....	77
Figure 6-31	Optimized TP transmission configuration	78
Figure 6-32	Trigger Transmit API for SoAd_IfTransmit configuration.....	79
Figure 6-33	NPduUdpTxQueue configuration	80
Figure 6-34	Event Queue limitation principle.....	80
Figure 6-35	Event Queue limitation configuration	81
Figure 6-36	Timeout List limitation configuration	81
Figure 6-37	PDU reception verification callback configuration	82
Figure 6-38	PDU reception verification socket connection configuration	82
Figure 6-39	Keep Socket Connection online configuration.....	83
Figure 6-40	Socket UDP Alive Supervision Timeout configuration	84
Figure 6-41	Socket TCP Keep Alive Time configuration	84
Figure 6-42	CDD configuration	85

Tables

Table 1-1	Component history.....	12
Table 3-1	Supported AUTOSAR standard conform feature.....	17

Table 3-2	Not supported AUTOSAR standard conform features	18
Table 3-3	Not supported AUTOSAR optional features	18
Table 3-4	Features provided beyond the AUTOSAR standard	18
Table 3-5	Service IDs	22
Table 3-6	Errors reported to DET	23
Table 4-1	Implementation files	24
Table 5-1	Type definitions	27
Table 5-2	SoAd_InitMemory	28
Table 5-3	SoAd_Init	28
Table 5-4	SoAd_IfTransmit	29
Table 5-5	SoAd_IfRoutingGroupTransmit	29
Table 5-6	SoAd_IfSpecificRoutingGroupTransmit	30
Table 5-7	SoAd_TpTransmit	30
Table 5-8	SoAd_Shutdown	31
Table 5-9	SoAd_TpCancelTransmit	31
Table 5-10	SoAd_TpCancelReceive	32
Table 5-11	SoAd_GetSoConId	33
Table 5-12	SoAd_OpenSoCon	33
Table 5-13	SoAd_CloseSoCon	34
Table 5-14	SoAd_RequestIpAddrAssignment	34
Table 5-15	SoAd_ReleaseIpAddrAssignment	35
Table 5-16	SoAd_GetLocalAddr	36
Table 5-17	SoAd_GetPhysAddr	36
Table 5-18	SoAd_GetRemoteAddr	37
Table 5-19	SoAd_GetRemoteAddrState	38
Table 5-20	SoAd_GetRcvRemoteAddr	38
Table 5-21	SoAd_EnableRouting	39
Table 5-22	SoAd_EnableSpecificRouting	39
Table 5-23	SoAd_DisableRouting	40
Table 5-24	SoAd_DisableSpecificRouting	41
Table 5-25	SoAd_SetRemoteAddr	41
Table 5-26	SoAd_SetUniqueRemoteAddr	42
Table 5-27	SoAd_ReadDhcpHostNameOption	42
Table 5-28	SoAd_WriteDhcpHostNameOption	43
Table 5-29	SoAd_ReadDhcpOption	44
Table 5-30	SoAd_WriteDhcpOption	44
Table 5-31	SoAd_GetVersionInfo	45
Table 5-32	SoAd_GetAndResetMeasurementData	46
Table 5-33	SoAd_MainFunctionRx	46
Table 5-34	SoAd_MainFunctionState	47
Table 5-35	SoAd_MainFunctionTx	47
Table 5-36	SoAd_MainFunction	48
Table 5-37	Services used by the Socket Adaptor	49
Table 5-38	SoAd_RxIndication	50
Table 5-39	SoAd_CopyTxData	50
Table 5-40	SoAd_TxConfirmation	51
Table 5-41	SoAd_LocalIpAddrAssignmentChg	52
Table 5-42	SoAd_TcpAccepted	52
Table 5-43	SoAd_TcpConnected	53
Table 5-44	SoAd_TcpIpEvent	53
Table 5-45	SoAd_DhcpEvent	54
Table 5-46	<Up>_[SoAd][If]RxIndication	55
Table 5-47	<Up>_[SoAd][If]TriggerTransmit	55
Table 5-48	<Up>_[SoAd][If]TxConfirmation	56

Table 5-49	<Up>_[SoAd][Tp]StartOfReception	56
Table 5-50	<Up>_[SoAd][Tp]CopyRxData	57
Table 5-51	<Up>_[SoAd][Tp]RxIndication	57
Table 5-52	<Up>_[SoAd][Tp]CopyTxData	58
Table 5-53	<Up>_[SoAd][Tp]TxConfirmation	59
Table 5-54	<Up>_SoConModeChg	59
Table 5-55	<Up>_LocalIpAddrAssignmentChg	60
Table 5-56	<Up>_ShutdownFinished	60
Table 5-57	<Up_TcpTlsSocketCreatedNotification>	61
Table 5-58	<Up_VerifyRxPdu>	61
Table 5-59	<Up_DhcpEvent>	62
Table 7-1	Glossary	86
Table 7-2	Abbreviations	87

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00	Created
1.01	BSD socket support (WinSock)
1.02	RAM variables moved into configuration
1.03	Spread buffer support (PbufType)
1.04	PduR integration
3.00	Extended functionality for IPv6
3.01	Support for UdpNm and other upper layer protocols DoIP: New header format for Routing Activation Request Call-back for EthSM
3.02	MISRA conformance
3.03	Changed IP address assignment call-back
3.04	DoIP extensions; dynamic UDP port usage
3.05	DaVinci Configurator Pro support; IPv6 support (DaVinci Configurator Pro only); Support of PDUR API according to AUTOSAR 4 for DoIP (DaVinci Configurator Pro only); Module optimization and clean-up
3.06	Possibility to choose various ISO13400 stages (DIS, FDIS, IS); Stubs for Cancel Transmit-/Receive and Change-/Read Parameter; Support of multiple additional include files for DoIP; RTE Service Ports to configure for VIN, GID, PowerMode Support of additional features XCP-Routing and Mirroring
3.07	Support of an additional customer specific feature IPv6 support for GENy configuration tool
4.00	Support of AUTOSAR 4.1 only with limited feature support
4.01	Extended functionality according to AUTOSAR 4.1
4.02	Support of streaming-based TCP TxConfirmation
4.03	Support of interfaces according to AUTOSAR 4.1.3
4.05	Service Discovery extensions; Support of Alive Supervision Timeout
4.07	Service Discovery extensions; Reworked Shutdown mechanism for upper layers (e.g. DoIP)
5.00	Complete extraction of DoIP
6.00	Support of major features of AUTOSAR 4.2.1
7.00	Optimized UDP retry behavior, Added BSD Socket API

Component Version	New Features
8.00	Support of configuration variant post-build loadable
8.01	Support of TLS client; Trigger Transmit API with SduLength In/Out
8.02	Release of BSD-Socket API
8.03	Extension of BSD Socket API to support SOME/IP-SD under Linux
9.00	MainFunction splitting; Optimized TP transmission; Trigger Transmit API for SoAd_IfTransmit; Optimized buffer handling for PDU fan-out
9.01	Event Queues and Timeout Lists
9.02	Support Buffer Size up to 128kB
10.00	Reworked header includes (P3 CAD)
10.01	Adapted API pattern (P3 Implementation API Pattern)
10.02	PDU reception verification (Callout for Diagnostic Firewall Use Case); “Transmission on specific socket connection” and “Forward socket connection on reception” (Support optimized PDU handling for C/S calls)
10.03	P3 Code Refactoring
10.04	P3 Code Refactoring / CDD Step 2
11.00	P3 Code Refactoring / CDD Step 3; SAFE Code Refactoring Support QNX
12.00	Support INTEGRITY
12.01	Support VLAN priorities for Linux
13.00	MISRA-C:2012
14.00	Forward of Tcplp_DhcpRead/WriteOption APIs to upper-layer
14.01	Bug fixing
15.00	Added DoIPInt to the supported modules
15.01	Added service to retrieve and reset measurement data
16.00	Support critical sections for reentrant transmission paths; Support for multi instance on client side using the same socket
16.01	Optimized IF transmission
16.02	SoAd PDU-Fanout for Somelp-TP
16.03	Finalize SoAd PDU-Fanout for Somelp-TP; Support meta data for TriggerTransmit
16.04	Make reception of broadcasts in SoAd configurable
16.05	Support keeping online of socket connections after transmission on automatic socket connection setup

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module Socket Adaptor as specified in [1].

Supported AUTOSAR Release*:	4.2	
Supported Configuration Variants:	pre-compile, post-build	
Vendor ID:	SOAD_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	SOAD_MODULE_ID	56 decimal (according to ref. [5])

* For the precise AUTOSAR Release 4.1.x please see the release specific documentation.

The Socket Adaptor provides communication between PDU based communication and socket based communication via TcpIp. Following key features are offered by the Socket Adaptor:

- > Support of TCP and UDP sockets over lower module TcpIp
- > Supports multiple socket connections per local socket to support multiple communication partners on the same local socket
- > Control API for socket connections or automated socket connection handling by Socket Adaptor
- > Independent reception (Socket Route) and transmission path (Pdu Route) on a socket connection
- > Support of Interface (IF) and Transport Protocol (TP) PDUs for upper layers
- > Generic upper layer configuration

Figure 2-1 provides a functional overview over Socket Adaptor and some examples of possible configuration variants.

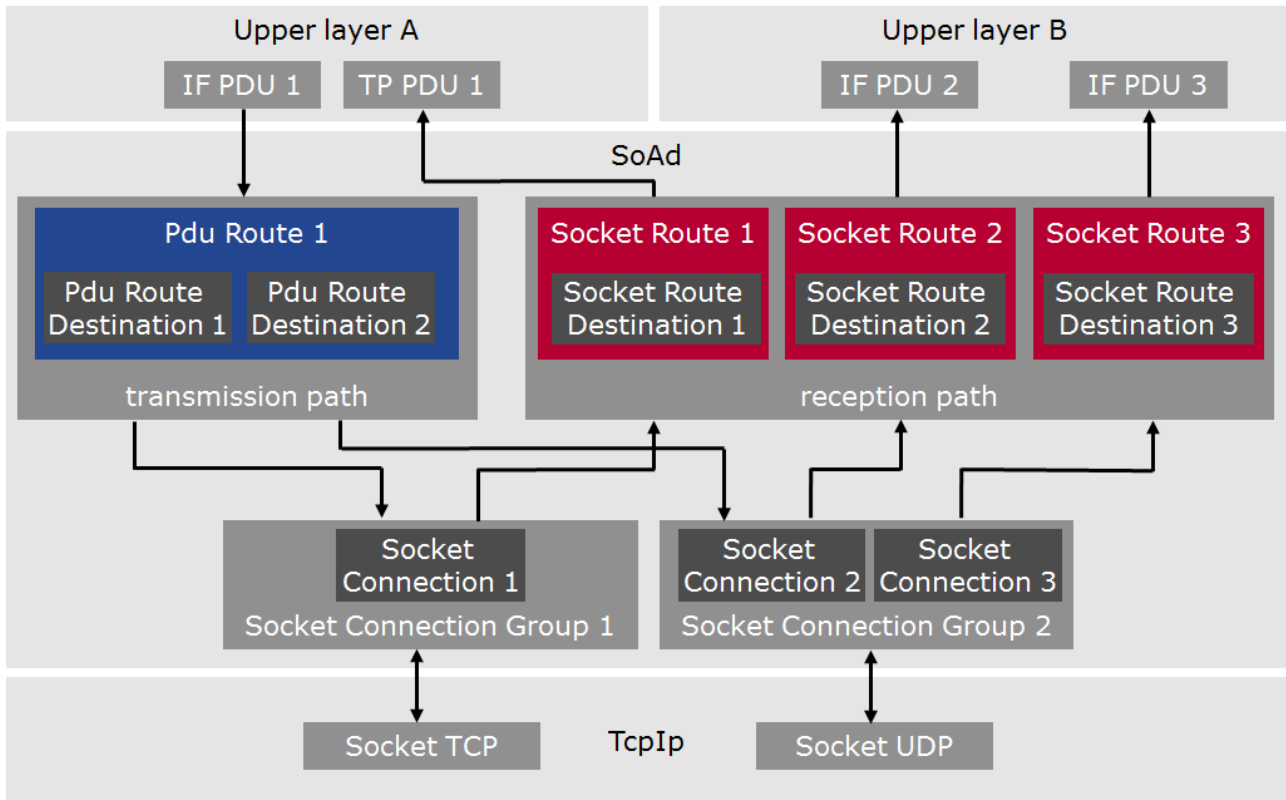


Figure 2-1 Functional Overview

2.1 Architecture Overview

The following figure shows where the Socket Adaptor is located in the AUTOSAR architecture.

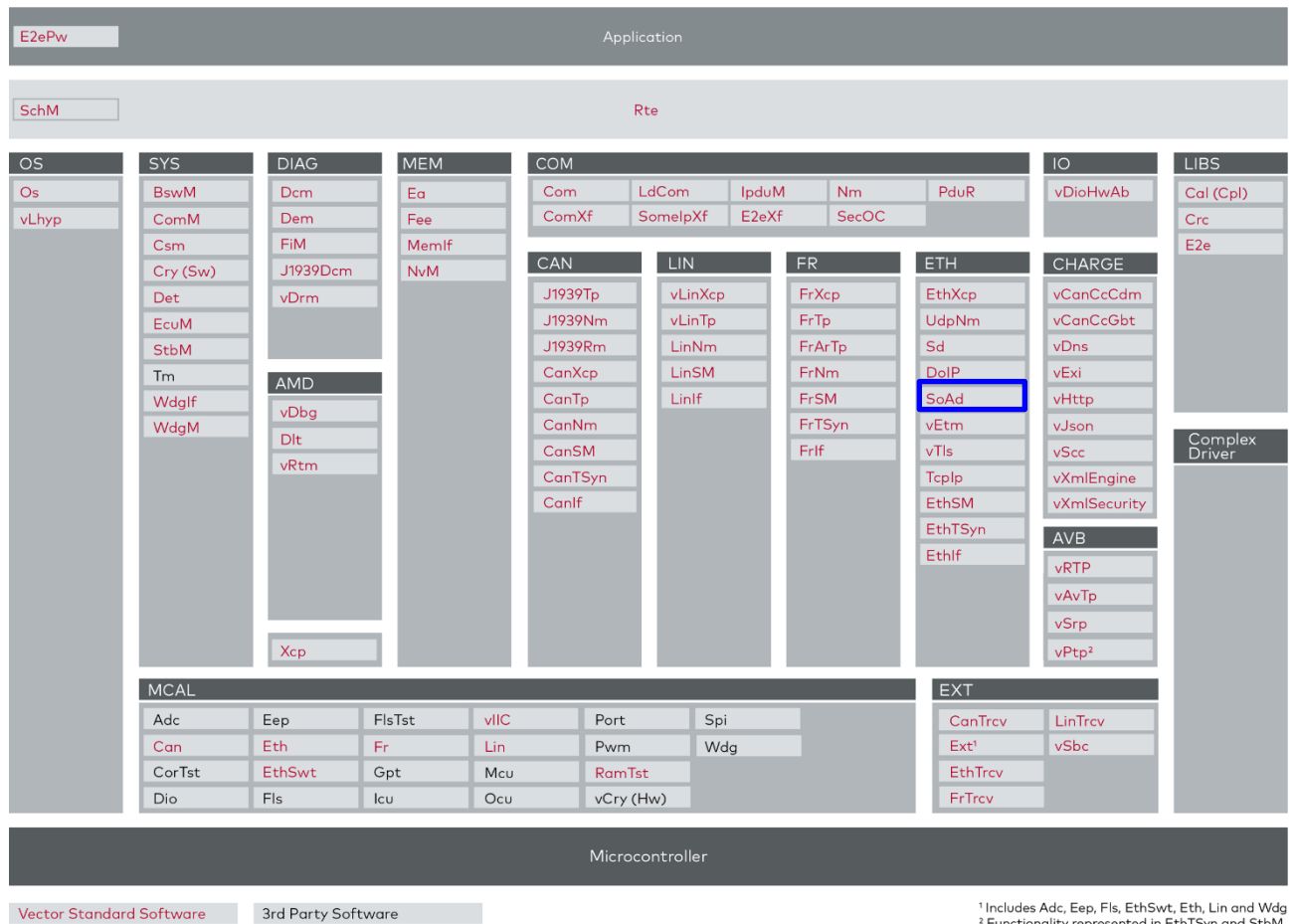


Figure 2-2 AUTOSAR 4.2 Architecture Overview

The next figure shows the interfaces of the Socket Adaptor. These interfaces are described in chapter 5.

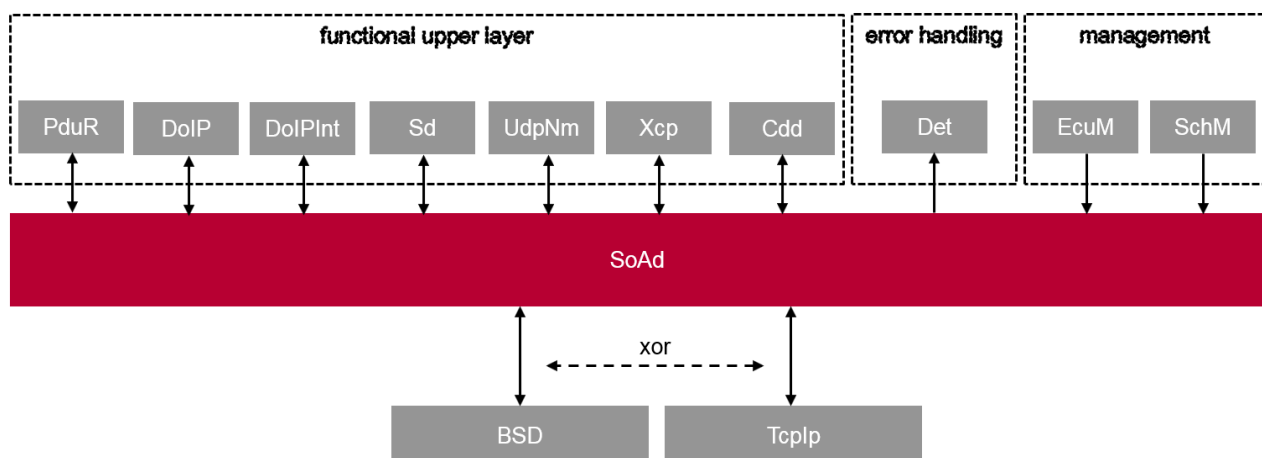


Figure 2-3 Interfaces to adjacent modules of the Socket Adaptor

Applications do not access the services of the BSW modules directly. They use the service ports provided by the BSW modules via the RTE. The Socket Adaptor does not support any service ports.

3 Functional Description

The features listed in the following tables cover the complete functionality specified for the Socket Adaptor.

The AUTOSAR standard functionality is specified in [1] or [2], the corresponding features are listed in the tables

- > Table 3-1 Supported AUTOSAR standard conform feature
- > Table 3-2 Not supported AUTOSAR standard conform features
- > Table 3-3 Not supported AUTOSAR optional features

Vector Informatik provides further Socket Adaptor functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

- > Table 3-4 Features provided beyond the AUTOSAR standard

3.1 Features

The following features specified in [1] or [2] are supported:

Supported AUTOSAR Standard Conform Features
Socket Connections and Socket Connection Groups
PDU Transmission
PDU Reception
PDU Header option
Best Match Algorithm
Message Acceptance Policy
TP PDU Cancelation
Disconnection and recovery
Routing Groups
PDU fan-out
Buffer handling (e.g. nPduUdpTxBuffer)
Error handling
Version check
Address assignment services
Support of post-build loadable
Get and reset measurement data

Table 3-1 Supported AUTOSAR standard conform feature

3.2 Deviations

The following features specified in [1] or [2] are not supported:

3.2.1 Not supported standard conform features

Not Supported AUTOSAR Standard Conform Features
Socket Routes (TCP/UDP) with multiple TP upper layers and disabled PDU Header option
Change Parameter service
Release remote IP address service

Table 3-2 Not supported AUTOSAR standard conform features

3.2.2 Not supported optional features

Not Supported AUTOSAR Standard Conform Features
Resource Management Option

Table 3-3 Not supported AUTOSAR optional features

3.2.3 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard.

Features Provided Beyond The AUTOSAR Standard
Best Match Algorithm with PDU Header validation
Best Match Algorithm with Socket Route validation
UDP immediate IF transmission confirmation (TxConfirmation)
API extension to get the remote address of received data (SoAd_GetRcvRemoteAddr)
Additional SoConModeChg notification
BSD Socket API for Linux, QNX and INTEGRITY (only for configuration variant pre-compile)
Shutdown mechanism
TLS client
MainFunction splitting
Optimized TP transmission
Trigger Transmit API for SoAd_IfTransmit
Optimized buffer handling for PDU fan-out
Event Queues and Timeout Lists
PDU reception verification
Read, write and events for DHCP options
Keeping online of socket connections with remote address set to wildcard after transmission on automatic socket connection setup

Table 3-4 Features provided beyond the AUTOSAR standard

**Caution**

There may be also some other deviations which are not documented here.

3.2.4 Known Issues (low priority)

3.2.4.1 ESCAN00087305

Restricted functionality of compiler abstraction

The compiler abstraction for pointers does always use the identical 'ptrclass', independently from the memory location of the target. (It is not differentiated between variables stored in the pre-compile or post-build memory sections.)

Hence, the compiler abstraction cannot be used to specify and optimize pointers (would lead to compiler errors).

Workaround: Do not use special optimizations in compiler abstraction.

3.2.5 Hints

3.2.5.1 CDD Contribution Type

Socket Adaptor supports the "CddSoAdUpperLayerContribution" with schema according to AUTOSAR 4.0.3. Older versions support "CddComIfUpperLayerContribution" instead of "CddSoAdUpperLayerContribution".

3.2.5.2 API deviation

The API to upper layer modules is implemented according to AUTOSAR 4.1.3 and partly to 4.2.1.

Please refer to chapter 5 for details.

3.2.5.3 UDP socket resources bound at startup

If a UDP socket connection remote address contains wildcards, socket connection can be opened on reception according to [1]. To support this feature corresponding socket connection must bind a TcpIp socket at ECU startup (i.e. first MainFunction cycle).

3.2.5.4 SoAd_SetUniqueRemoteAddress() disables alive supervision timeout

If `SoAd_SetUniqueRemoteAddress()` is called for a UDP socket connection group and a corresponding socket connection in state online is found, alive supervision timeout will be disabled for this socket connection.

3.2.5.5 SoAd_CloseSoCon() if open/close counter is 0

If `SoAd_CloseSoCon()` is called with parameter `abort` set to `TRUE` and open/close counter is 0, caused by socket connection open in reception of data, the corresponding socket connection will be closed anyway. If parameter `abort` is set to `FALSE`, socket connection is not closed.

This behavior was implemented to close socket connections by user in all cases and to prevent always open socket connections that blocks communication with other remote entities.

3.3 Initialization

The Socket Adaptor is initialized via a `SoAd_InitMemory()` call followed by call of `SoAd_Init()`.

**Example**

```
SoAd_Init(SoAd_Config_Ptr);
```

3.3.1 Configuration Variants 1, 2 (Pre-Compile and Link-Time)

At configuration Variant 1 (Pre-compile) and Variant 2 (Link-Time) the SoAd module has to be initialized using the `SoAd_Init()` function with the address of the pre-compile configuration data passed as parameter. The declaration of the pre-compile configuration data is contained in the files `SoAd_Lcfg.h` and `SoAd_Lcfg.c`.

3.3.2 Configuration Variant 3 (Post-build)

In this configuration Variant, the SoAd module has to be initialized using the `SoAd_Init()` function with the address of the post-build configuration data passed as parameter. The declaration of the post-build configuration data is contained in the files `SoAd_PBcfg.h` and `SoAd_PBcfg.c`.

Please refer to chapter 6.1 to get information about supported configuration variants.

3.4 States

The Socket Adaptor has an extended state handling after calling the initialization functions (described in chapter before). Figure 3-1 shows the states of Socket Adaptor when using the shutdown feature described in 6.2.4.6.

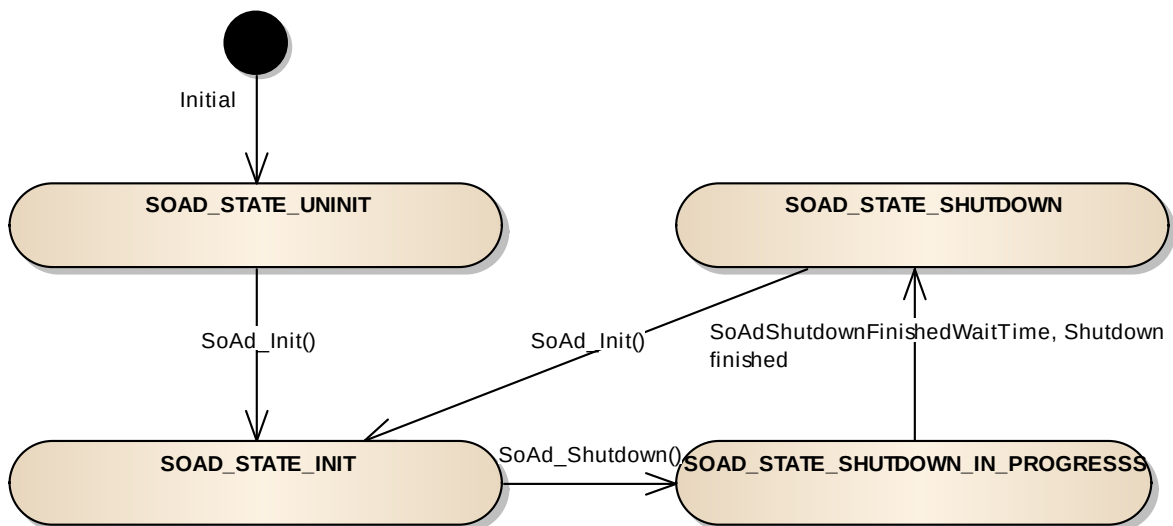


Figure 3-1 Module states

3.5 Main Functions

The Socket Adaptor has one main function (except when 6.2.4.8 MainFunction splitting is enabled) which handles

- > Transmission/reception and socket handling on BSD Socket API
- > Socket connection state handling
- > TP transmission/reception
- > TP transmission/reception cancellation
- > UDP nPduUdpTxBuffer
- > TriggerTransmit transmission
- > Handle pending transmission confirmation

3.6 Error Handling

3.6.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [3], if development error reporting is enabled (i.e. pre-compile parameter `SOAD_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator but must have the same signature as the service `Det_ReportError()`.

The reported Socket Adaptor ID is 56.

The reported service IDs identify the services which are described in 5.2. The following table presents the service IDs and the related services:

Service ID	Service
0x01	SOAD_SID_INIT
0x02	SOAD_SID_GET_VERSION_INFO
0x03	SOAD_SID_IF_TRANSMIT
0x04	SOAD_SID_TP_TRANSMIT
0x05	SOAD_SID_TP_CANCEL_TRANSMIT
0x06	SOAD_SID_TP_CANCEL_RECEIVE
0x07	SOAD_SID_GET_SO_CON_ID
0x08	SOAD_SID_OPEN_SO_CON
0x09	SOAD_SID_CLOSE_SO_CON
0x0A	SOAD_SID_REQ_IP_ADDR_ASSIGN
0x0B	SOAD_SID_RLS_IP_ADDR_ASSIGN
0x0C	SOAD_SID_GET_LOCAL_ADDR

Service ID	Service
0x0D	SOAD_SID_GET_PHYS_ADDR
0x0E	SOAD_SID_ENABLE_ROUTING
0x0F	SOAD_SID_DISABLE_ROUTING
0x10	SOAD_SID_SET_REMOTE_ADDR
0x11	SOAD_SID_TP_CHANGE_PARAMETER
0x12	SOAD_SID_RX_INDICATION
0x13	SOAD_SID_COPY_TX_DATA
0x14	SOAD_SID_TX_CONFIRMATION
0x15	SOAD_SID_TCP_ACCEPTED
0x16	SOAD_SID_TCP_CONNECTED
0x17	SOAD_SID_TCPIP_EVENT
0x18	SOAD_SID_LOCAL_IP_ADDR_ASSIGNMENT_CHG
0x19	SOAD_SID_MAIN_FUNCTION
0x1A	SOAD_SID_READ_DHCP_HOST_NAME_OPT
0x1B	SOAD_SID_WRITE_DHCP_HOST_NAME_OPT
0x1C	SOAD_SID_GET_REMOTE_ADDR
0x1D	SOAD_SID_IF_ROUT_GROUP_TRANSMIT
0x1E	SOAD_SID_SET_UNI_REMOTE_ADDR
0x1F	SOAD_SID_IF_SPEC_ROUT_GROUP_TRANSMIT
0x20	SOAD_SID_ENABLE_SPECIFIC_ROUTING
0x21	SOAD_SID_DISABLE_SPECIFIC_ROUTING
0x45	SOAD_SID_GET_RESET_MEASURE_DATA
0xD0	SOAD_SID_MAIN_FUNCTION_RX
0xD1	SOAD_SID_MAIN_FUNCTION_STATE
0xD2	SOAD_SID_MAIN_FUNCTION_TX
0xD3	SOAD_SID_SHUTDOWN
0xD4	SOAD_SID_GET_RCV_REMOTE_ADDR
0xD5	SOAD_SID_GET_REMOTE_ADDR_STATE
0xD6	SOAD_SID_READ_DHCP_OPT
0xD7	SOAD_SID_WRITE_DHCP_OPT
0xD8	SOAD_SID_DHCP_EVENT

Table 3-5 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x01	SOAD_E_NOTINIT
0x02	SOAD_E_PARAM_POINTER
0x03	SOAD_E_INV_ARG
0x04	SOAD_E_NOBUFS
0x05	SOAD_E_INV_PDUHEADER_ID
0x06	SOAD_E_INV_PDUID
0x07	SOAD_E_INV_SOCKETID
0x08	SOAD_E_INIT_FAILED

Table 3-6 Errors reported to DET

3.6.2 Production Code Error Reporting

By default, production code related errors are reported to the DEM using the service `Dem_ReportErrorStatus()` as specified in [4].

The Socket Adaptor does not support DEM errors.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR Socket Adaptor into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the Socket Adaptor consists out of these files:

File Name	Description
SoAd.c	Static source file
SoAd.h	Static header file
SoAd_Cbk.h	Static header file for callback functions
SoAd_EventQueue.c	Static source file for sub-module which handles event queues
SoAd_EventQueue.h	Static header file for sub-module which handles event queues
SoAd_Measure.c	Static source file for sub-module which handles measurement data
SoAd_Measure.h	Static header file for sub-module which handles measurement data
SoAd_Priv.h	Static header file for Socket Adaptor internal usage
SoAd_RouteGrp.c	Static source file for sub-module which handles routing groups
SoAd_RouteGrp.h	Static header file for sub-module which handles routing groups
SoAd_Rx.c	Static source file for sub-module which handles reception
SoAd_Rx.h	Static header file for sub-module which handles reception
SoAd_SoCon.c	Static source file for sub-module which handles socket connections
SoAd_SoCon.h	Static header file for sub-module which handles socket connections
SoAd_Tcplp.c	Static source file for sub-module which handles Tcplp-Stack specifics
SoAd_Tcplp.h	Static header file for sub-module which handles Tcplp-Stack specifics
SoAd_TimeoutList.c	Static source file for sub-module which handles timeout lists
SoAd_TimeoutList.h	Static header file for sub-module which handles timeout lists
SoAd_Tx.c	Static source file for sub-module which handles transmission
SoAd_Tx.h	Static header file for sub-module which handles transmission
SoAd_Util.c	Static source file for sub-module which provides general operations
SoAd_Util.h	Static header file for sub-module which provides general operations
SoAd_Types.h	Static header file containing types
SoAd_Lcfg.c	Generated source file (e.g. RAM/ROM mapping tables)
SoAd_Lcfg.h	Generated header file
SoAd_Cfg.h	Generated header file for configuration parameter (e.g. feature switches)
SoAd_PBcfg.c	Generated source file (Post-build configuration)
SoAd_PBcfg.h	Generated header file (Post-build configuration)
SoAd.lib	Library if Socket Adaptor is not delivered with source code

Table 4-1 Implementation files

4.2 Critical Sections

All services and callbacks for transmission, reception and state changes of Socket Adaptor may be called in interrupt or task level. Thus a synchronization mechanism is implemented to guarantee data consistency.

The synchronization mechanism defined by AUTOSAR covers the entering and leaving of so called critical sections.

The implementation of the critical sections must avoid that multiple relevant tasks or interrupt service routines can enter each of the critical sections more than once at the same time.

Relevant interrupt services in the Socket Adaptor context are interrupt services originated from physical bus events (Ethernet, CAN, LIN, FlexRay etc.).

Relevant tasks in the Socket Adaptor context are all tasks which call Socket Adaptor API functions. Usually these tasks are limited to tasks on which other BSW modules (DoIP, Sd, Xcp etc.) are mapped to.

A critical section can be handled by using the so called “Exclusive Areas”. The Socket Adaptor defines the following exclusive areas:

- > SOAD_EXCLUSIVE_AREA_0 is used whenever memory accesses must be protected from accesses of interrupting calls to services and callbacks of Socket Adaptor. This exclusive area may be entered in interrupt or task context. The frequency of entering and leaving this area will be very high. The average length of stay in the area is medium.
- > SOAD_EXCLUSIVE_AREA_1 is used in main function context to prevent that Socket Adaptor internal transmissions (only nPdu and routing group transmission) are interrupted by transmission requests of any user. The transmission requests are rejected in that case. This may lead to data loss if e.g. no retry mechanism is implemented by the SoAd users. The frequency of entering and leaving this area will be medium. The average length of stay in the area is undefined but expected to be large since the entire transmission path is protected.

For an implementation of the critical sections it could be sufficient to:

- > Disable all bus relevant interrupts of all buses related to calls to Socket Adaptor API functions (e.g. gateway use-case).
- > Disable all Ethernet bus relevant interrupts if all modules calling Socket Adaptor API functions are mapped to one task (e.g. SchM task) or a non-preemptive OS is used.
- > Not implement the critical section in case BSD Socket API is used and the calls to Socket Adaptor API do not interrupt the task of Socket Adaptor.

Please note that these are only examples and that the actual implementation of the critical sections is highly dependent on the platform architecture and the system configuration.

4.3 Main Functions

4.3.1 BSD Socket API

In case BSD Socket API is used, some operations might take longer than with the AUTOSAR TCP/IP. This can influence the execution time of the SoAd main function. Therefore, an increased main function cycle might be required to avoid multiple task activation and to ensure a stable periodic call of the SoAd main function.

4.3.2 Preemption

SoAd expects that its own main functions do not interrupt each other. SoAd expects that an own main function does not interrupt the main functions of related modules and is not interrupted by main functions of related modules.

The SoAd related module is the Tcplp module.



Caution

Consider main function expectations when using preemptive tasks.

Additionally, please consider the usage of critical sections as described in chapter 4.2

5 API Description

For an interfaces overview please see Figure 2-3.

5.1 Type Definitions

The types defined by the Socket Adaptor are described in this chapter .

Type Name	C-Type	Description	Value Range
SoAd_RemAddrStateType	uint8	Describes remote IP address and port state.	SOAD_SOCON_IP_SET_PORT_SET
			SOAD_SOCON_IP_SET_PORT_ANY
			SOAD_SOCON_IP_SET_PORT_NOT
			SOAD_SOCON_IP_ANY_PORT_SET
			SOAD_SOCON_IP_ANY_PORT_ANY
			SOAD_SOCON_IP_ANY_PORT_NOT
			SOAD_SOCON_IP_NOT_PORT_SET
			SOAD_SOCON_IP_NOT_PORT_ANY
			SOAD_SOCON_IP_NOT_PORT_NOT
SoAd_MeasurementIdxType	uint8	Index to select specific measurement data	SOAD_MEAS_DROP_TCP
			SOAD_MEAS_DROP_UDP
			SOAD_MEAS_DROP_TCP_CONNECTION
			SOAD_MEAS_DROP_UDP_FRAME
			SOAD_MEAS_ALL

Table 5-1 Type definitions

5.2 Services provided by Socket Adaptor

This chapter describes the service functions that are implemented by the Socket Adaptor and can be invoked by other modules. The prototypes of the service functions are provided in the header file `SoAd.h` by the Socket Adaptor.

5.2.1 SoAd_InitMemory

Prototype	
<code>void SoAd_InitMemory (void)</code>	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initializes *_INIT_*-variables.	
Service to initialize module global variables at power up. This function initializes the variables in *_INIT_* sections. Used in case they are not initialized by the startup code.	

Particularities and Limitations	
Module is uninitialized.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-2 SoAd_InitMemory

5.2.2 SoAd_Init

Prototype	
void SoAd_Init (const SoAd_ConfigType *SoAdConfigPtr)	
Parameter	
SoAdConfigPtr [in]	Configuration structure for initializing the module.
Return code	
void	none
Functional Description	
Initializes module. This function initializes the module SoAd. It initializes all variables and sets the module state to initialized.	
Particularities and Limitations	
> Interrupts are disabled. SoAd_InitMemory has been called unless SoAd_ModuleInitialized is initialized by start-up code.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-3 SoAd_Init

5.2.3 SoAd_IfTransmit

Prototype	
Std_ReturnType SoAd_IfTransmit (PduIdType SoAdSrcPduId, const PduInfoType *SoAdSrcPduInfoPtr)	
Parameter	
SoAdSrcPduId [in]	Tx PDU identifier.
SoAdSrcPduInfoPtr [in]	Pointer to PDU.
Return code	
Std_ReturnType	E_OK Transmit request was accepted.

Std_ReturnType	E_NOT_OK Transmit request was not accepted.
Functional Description	
Transmits an IF-PDU.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-4 SoAd_IfTransmit

5.2.4 SoAd_IfRoutingGroupTransmit

Prototype	
Std_ReturnType SoAd_IfRoutingGroupTransmit (SoAd_RoutingGroupIdType id)	
Parameter	
id [in]	Routing group identifier.
Return code	
Std_ReturnType	E_OK Transmit request was accepted.
Std_ReturnType	E_NOT_OK Transmit request was not accepted.
Functional Description	
Triggers transmission of all IF-PDUs related to a routing group.	
Triggers transmission via trigger transmit in main function context.	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-5 SoAd_IfRoutingGroupTransmit

5.2.5 SoAd_IfSpecificRoutingGroupTransmit

Prototype	
Std_ReturnType SoAd_IfSpecificRoutingGroupTransmit (SoAd_RoutingGroupIdType id, SoAd_SoConIdType SoConId)	
Parameter	
id [in]	Routing group identifier.

SoConId [in]	Socket connection identifier.
Return code	
Std_ReturnType	E_OK Transmit request was accepted.
Std_ReturnType	E_NOT_OK Transmit request was not accepted.
Functional Description	
Triggers transmission of all IF-PDUs related to a routing group and socket connection. Triggers transmission via trigger transmit in main function context.	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-6 SoAd_IfSpecificRoutingGroupTransmit

5.2.6 SoAd_TpTransmit

Prototype	
Std_ReturnType SoAd_TpTransmit (PduIdType SoAdSrcPduId, const PduInfoType *SoAdSrcPduInfoPtr)	
Parameter	
SoAdSrcPduId [in]	Tx PDU identifier.
SoAdSrcPduInfoPtr [in]	Pointer to PDU (length is evaluated only).
Return code	
Std_ReturnType	E_OK Transmit request was accepted.
Std_ReturnType	E_NOT_OK Transmit request was not accepted.
Functional Description	
Transmits a TP-PDU. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-7 SoAd_TpTransmit

5.2.7 SoAd_Shutdown

Prototype	
Std_ReturnType SoAd_Shutdown (void)	
Parameter	
void	none
Return code	
Std_ReturnType	E_OK Shutdown request was accepted.
	SOAD_E_INPROGRESS Shutdown is in progress.
	E_NOT_OK Shutdown request was not accepted.
Functional Description	
Shuts down SoAd module.	
Closes all open socket connections and disables transmission and reception.	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Non-Reentrant	

Table 5-8 SoAd_Shutdown

5.2.8 SoAd_TpCancelTransmit

Prototype	
Std_ReturnType SoAd_TpCancelTransmit (PduIdType PduId)	
Parameter	
PduId [in]	Tx PDU identifier.
Return code	
Std_ReturnType	E_OK Transmit cancellation request was accepted.
	E_NOT_OK Transmit cancellation request was not accepted.
Functional Description	
Requests transmission cancellation of a specific TP-PDU.	
-	
Particularities and Limitations	
Transmission of PDU is requested via SoAd_TpTransmit.	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-9 SoAd_TpCancelTransmit

5.2.9 SoAd_TpCancelReceive

Prototype	
Std_ReturnType SoAd_TpCancelReceive (PduIdType PduId)	
Parameter	
PduId [in]	Rx PDU identifier.
Return code	
Std_ReturnType	E_OK Receive cancellation request was accepted.
Std_ReturnType	E_NOT_OK Receive cancellation request was not accepted.
Functional Description	
Requests reception cancellation of a specific TP-PDU.	
-	
Particularities and Limitations	
Reception of PDU is initiated via <Up>_[SoAd][Tp]StartOfReception.	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-10 SoAd_TpCancelReceive

5.2.10 SoAd_GetSoConId

Prototype	
Std_ReturnType SoAd_GetSoConId (PduIdType TxPduId, SoAd_SoConIdType *SoConIdPtr)	
Parameter	
TxPduId [in]	Tx PDU identifier.
SoConIdPtr [out]	Pointer to the socket connection identifier.
Return code	
Std_ReturnType	E_OK Socket connection identifier was found.
Std_ReturnType	E_NOT_OK Socket connection identifier was not found.
Functional Description	
Returns the socket connection identifier of a specific Tx PDU identifier.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous	

> This function is Reentrant

Table 5-11 SoAd_GetSoConId

5.2.11 SoAd_OpenSoCon

Prototype	
Std_ReturnType SoAd_OpenSoCon (SoAd_SoConIdType SoConId)	
Parameter	
SoConId [in]	Socket connection identifier.
Return code	
Std_ReturnType	E_OK Open request was accepted.
Std_ReturnType	E_NOT_OK Open request was not accepted.
Functional Description	
Opens a socket connection. Opens the socket connection in context of main function.	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-12 SoAd_OpenSoCon

5.2.12 SoAd_CloseSoCon

Prototype	
Std_ReturnType SoAd_CloseSoCon (SoAd_SoConIdType SoConId, boolean Abort)	
Parameter	
SoConId [in]	Socket connection identifier.
Abort [in]	Flag to close socket connection immediately. [range: TRUE close immediately, FALSE close when open close sequence is 0]
Return code	
Std_ReturnType	E_OK Close request was accepted.
Std_ReturnType	E_NOT_OK Close request was not accepted.
Functional Description	
Closes a socket connection. Closes the socket connection in context of main function.	
Particularities and Limitations	
-	

Call context
> TASK ISR2 > This function is Reentrant

Table 5-13 SoAd_CloseSoCon

5.2.13 SoAd_RequestIpAddrAssignment

Prototype	
<pre>Std_ReturnType SoAd_RequestIpAddrAssignment (SoAd_SoConIdType SoConId, SoAd_IpAddrAssignmentType Type, SoAd_SockAddrType *LocalIpAddrPtr, uint8 Netmask, SoAd_SockAddrType *DefaultRouterPtr)</pre>	
Parameter	
SoConId [in]	Socket connection identifier.
Type [in]	IP address type. [range in case of SOAD_SOCKET_API != SOAD_SOCKET_API_AUTOSAR: SOAD_IPADDR_ASSIGNMENT_STATIC .. SOAD_IPADDR_ASSIGNMENT_IPV6_ROUTER]
LocalIpAddrPtr [in]	Pointer to IP address which shall be assigned. [Points to one of the following structs depending on value of struct element domain: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
Netmask [in]	Netmask in CIDR.
DefaultRouterPtr [in]	Pointer to default router (gateway) address. [Type == SOAD_IPADDR_ASSIGNMENT_STATIC: → DefaultRouterPtr != NULL_PTR Points to one of the following structs depending on configured IP address version of parameter SoConId: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6 Type != SOAD_IPADDR_ASSIGNMENT_STATIC: → DefaultRouterPtr == NULL_PTR]
Return code	
Std_ReturnType	E_OK Assignment request was accepted.
Std_ReturnType	E_NOT_OK Assignment request was not accepted.
Functional Description	
Requests IP address assignment on a local address identified by a socket connection.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-14 SoAd_RequestIpAddrAssignment

5.2.14 SoAd_ReleaseIpAddrAssignment

Prototype	
Std_ReturnType SoAd_ReleaseIpAddrAssignment (SoAd_SoConIdType SoConId)	
Parameter	
SoConId [in]	Socket connection identifier.
Return code	
Std_ReturnType	E_OK Release request was accepted.
Std_ReturnType	E_NOT_OK Release request was not accepted.
Functional Description	
Releases IP address assignment on a local address identified by a socket connection.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-15 SoAd_ReleaseIpAddrAssignment

5.2.15 SoAd_GetLocalAddr

Prototype	
Std_ReturnType SoAd_GetLocalAddr (SoAd_SoConIdType SoConId, SoAd_SockAddrType *LocalAddrPtr, uint8 *NetmaskPtr, SoAd_SockAddrType *DefaultRouterPtr)	
Parameter	
SoConId [in]	Socket connection identifier.
LocalAddrPtr [out]	Pointer to local address (IP and Port). [Points to one of the following structs depending on value of struct element domain: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
NetmaskPtr [out]	Pointer to network mask (CIDR Notation).
DefaultRouterPtr [out]	Pointer to default router (gateway). [Points to one of the following structs depending on configured IP address version of parameter SoConId: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.

Functional Description
Returns a local IP address identified by a socket connection.
-
Particularities and Limitations
-
Call context
> TASK ISR2 > This function is Synchronous > This function is Reentrant

Table 5-16 SoAd_GetLocalAddr

5.2.16 SoAd_GetPhysAddr

Prototype	
Std_ReturnType SoAd_GetPhysAddr (SoAd_SoConIdType SoConId, uint8 *PhysAddrPtr)	
Parameter	
SoConId [in]	Socket connection identifier.
PhysAddrPtr [out]	Pointer to physical address. [Points to a uint8 array of length 6]
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Returns the physical address (MAC address) of a local interface identified by a socket connection.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-17 SoAd_GetPhysAddr

5.2.17 SoAd_GetRemoteAddr

Prototype
Std_ReturnType SoAd_GetRemoteAddr (SoAd_SoConIdType SoConId, SoAd_SockAddrType *IpAddrPtr)

Parameter	
SoConId [in]	Socket connection identifier.
IpAddrPtr [out]	Pointer to remote address. [Points to one of the following structs depending on configured IP address version of parameter SoConId: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Returns the remote address of a socket connection. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-18 SoAd_GetRemoteAddr

5.2.18 SoAd_GetRemoteAddrState

Prototype	
Std_ReturnType SoAd_GetRemoteAddrState (SoAd_SoConIdType SoConId, SoAd_SockAddrType *IpAddrPtr, SoAd_RemAddrStateType *RemAddrState)	
Parameter	
SoConId [in]	Socket connection identifier.
IpAddrPtr [out]	Pointer to remote address. [Points to one of the following structs depending on configured IP address version of parameter SoConId: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
RemAddrState [out]	Pointer to remote address state.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Returns the remote address and remote address state of a socket connection. -	

Particularities and Limitations
-
Call context
> TASK ISR2 > This function is Synchronous > This function is Reentrant

Table 5-19 SoAd_GetRemoteAddrState

5.2.19 SoAd_GetRcvRemoteAddr

Prototype	
Std_ReturnType SoAd_GetRcvRemoteAddr (SoAd_SoConIdType SoConId, SoAd_SockAddrType *IpAddrPtr)	
Parameter	
SoConId [in]	Socket connection identifier.
IpAddrPtr [out]	Pointer to remote address. [Points to one of the following structs depending on configured IP address version of parameter SoConId: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Returns the remote address of the last received message on a socket connection.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-20 SoAd_GetRcvRemoteAddr

5.2.20 SoAd_EnableRouting

Prototype	
Std_ReturnType SoAd_EnableRouting (SoAd_RoutingGroupIdType id)	
Parameter	
id [in]	Routing group identifier.

Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Enables a routing group. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-21 SoAd_EnableRouting

5.2.21 SoAd_EnableSpecificRouting

Prototype	
Std_ReturnType SoAd_EnableSpecificRouting (SoAd_RoutingGroupIdType id, SoAd_SoConIdType SoConId)	
Parameter	
id [in]	Routing group identifier.
SoConId [in]	Socket connection identifier.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Enables a routing group on a specific socket connection. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-22 SoAd_EnableSpecificRouting

5.2.22 SoAd_DisableRouting

Prototype	
Std_ReturnType SoAd_DisableRouting (SoAd_RoutingGroupIdType id)	
Parameter	
id [in]	Routing group identifier.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Disables a routing group. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-23 SoAd_DisableRouting

5.2.23 SoAd_DisableSpecificRouting

Prototype	
Std_ReturnType SoAd_DisableSpecificRouting (SoAd_RoutingGroupIdType id, SoAd_SoConIdType SoConId)	
Parameter	
id [in]	Routing group identifier.
SoConId [in]	Socket connection identifier.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Disables a routing group on a specific socket connection. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous	

> This function is Reentrant

Table 5-24 SoAd_DisableSpecificRouting

5.2.24 SoAd_SetRemoteAddr

Prototype	
Std_ReturnType SoAd_SetRemoteAddr (SoAd_SoConIdType SoConId, SoAd_SockAddrType *RemoteAddrPtr)	
Parameter	
SoConId [in]	Socket connection identifier.
RemoteAddrPtr [in]	Pointer to remote address. [Points to one of the following structs depending on configured IP address version of parameter SoConId: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Sets the remote address of a socket connection. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-25 SoAd_SetRemoteAddr

5.2.25 SoAd_SetUniqueRemoteAddr

Prototype	
Std_ReturnType SoAd_SetUniqueRemoteAddr (SoAd_SoConIdType SoConId, const SoAd_SockAddrType *RemoteAddrPtr, SoAd_SoConIdType *AssignedSoConIdPtr)	
Parameter	
SoConId [in]	Socket connection identifier (any socket connection in socket connection group).
RemoteAddrPtr [in]	Pointer to remote address. [Points to one of the following structs depending on configured IP address version of parameter SoConId: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]

AssignedSoConIdPtr [out]	Pointer to assigned socket connection identifier.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Sets the remote address of a suitable socket connection in a socket connection group. Considers the best match algorithm to select the socket connection.	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-26 SoAd_SetUniqueRemoteAddr

5.2.26 SoAd_ReadDhcpHostNameOption

Prototype	
Std_ReturnType SoAd_ReadDhcpHostNameOption (SoAd_SoConIdType SoConId, uint8 *length, uint8 *data)	
Parameter	
SoConId [in]	Socket connection identifier.
length [in,out]	Length of buffer for hostname (length of provided buffer, updated to length of hostname).
data [out]	Pointer to buffer for hostname.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Returns the DHCP hostname option currently configured on a local interface identified by a socket connection. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-27 SoAd_ReadDhcpHostNameOption

5.2.27 SoAd_WriteDhcpHostNameOption

Prototype	
Std_ReturnType SoAd_WriteDhcpHostNameOption (SoAd_SoConIdType SoConId, uint8 length, const uint8 *data)	
Parameter	
SoConId [in]	Socket connection identifier.
length [in]	Length of buffer for hostname.
data [in]	Pointer to buffer for hostname.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Sets the DHCP hostname option on a local interface identified by a socket connection. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-28 SoAd_WriteDhcpHostNameOption

5.2.28 SoAd_ReadDhcpOption

Prototype	
Std_ReturnType SoAd_ReadDhcpOption (SoAd_LocalAddrIdType IpAddrId, uint16 Option, uint16 *DataLengthPtr, uint8 *DataPtr)	
Parameter	
IpAddrId [in]	IP address identifier.
Option [in]	DHCP option code.
DataLengthPtr [in,out]	Pointer to length of option buffer (updated to length of option on return).
DataPtr [out]	Pointer to option buffer.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Returns a DHCP option on a local address.	

-
Particularities and Limitations
-
Call context
> TASK ISR2
> This function is Synchronous
> This function is Reentrant

Table 5-29 SoAd_ReadDhcpOption

5.2.29 SoAd_WriteDhcpOption

Prototype	
Std_ReturnType SoAd_WriteDhcpOption (SoAd_LocalAddrIdType IpAddrId, uint16 Option, uint16 DataLength, const uint8 *DataPtr)	
Parameter	
IpAddrId [in]	IP address identifier.
Option [in]	DHCP option code.
DataLength [in]	Length of option buffer.
DataPtr [in]	Pointer to option buffer.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Sets a DHCP option on a local address.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2	
> This function is Synchronous	
> This function is Reentrant	

Table 5-30 SoAd_WriteDhcpOption

5.2.30 SoAd_GetVersionInfo

Prototype	
void SoAd_GetVersionInfo (Std_VersionInfoType *versioninfo)	

Parameter	
versioninfo [out]	Pointer to where to store the version information. Parameter must not be NULL.
Return code	
void	none
Functional Description	
Returns the version information. Returns version information, vendor ID and AUTOSAR module ID of the component.	
Particularities and Limitations	
-	
Configuration Variant(s): SOAD_VERSION_INFO_API	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-31 SoAd_GetVersionInfo

5.2.31 SoAd_GetAndResetMeasurementData

Prototype	
void SoAd_GetAndResetMeasurementData (SoAd_MeasurementIdxType MeasurementIdx, boolean MeasurementResetNeeded, uint32 *MeasurementDataPtr)	
Parameter	
MeasurementIdx [in]	The index to select specific measurement data. [range: SOAD_MEAS_DROP_TCP, SOAD_MEAS_DROP_UDP, SOAD_MEAS_DROP_TCP_CONNECTION, SOAD_MEAS_DROP_UDP_FRAME, SOAD_MEAS_ALL]
MeasurementResetNeeded [in]	Flag to indicate if the counter needs to be reset. [range: TRUE, FALSE]
MeasurementDataPtr [out]	Buffer where the value of the counter is to be copied into. [range: POINTER may be NULL_PTR]
Return code	
Std_ReturnType	E_OK The operations were successful.
Std_ReturnType	E_NOT_OK The operations failed.
Functional Description	
Gets and Resets (if requested) the measurement data. Gets and Resets (if requested) the value of the counter of dropped TCP/UDP packets, dropped TCP connections and dropped UDP frames.	
Particularities and Limitations	
-	

Configuration Variant(s): SOAD_GET_RESET_MEASUREMENT_DATA_API
Call context
> TASK ISR2 > This function is Synchronous > This function is Reentrant

Table 5-32 SoAd_GetAndResetMeasurementData

5.2.32 SoAd_MainFunctionRx

Prototype	
void SoAd_MainFunctionRx (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Schedules the Socket Adaptor (Entry point for scheduling) and handles asynchronous reception.	
Particularities and Limitations	
- -	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-33 SoAd_MainFunctionRx

5.2.33 SoAd_MainFunctionState

Prototype	
void SoAd_MainFunctionState (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Schedules the Socket Adaptor (Entry point for scheduling) and handles states.	
Particularities and Limitations	
- -	
Call context	

- > TASK
- > This function is Synchronous
- > This function is Non-Reentrant

Table 5-34 SoAd_MainFunctionState

5.2.34 SoAd_MainFunctionTx

Prototype	
void SoAd_MainFunctionTx (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Schedules the Socket Adaptor (Entry point for scheduling) and handles asynchronous transmission.	
Particularities and Limitations	
-	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-35 SoAd_MainFunctionTx

5.2.35 SoAd_MainFunction

Prototype	
void SoAd_MainFunction (void)	
Parameter	
void	none
Return code	
void	none
Functional Description	
Schedules the Socket Adaptor (Entry point for scheduling) and handles asynchronous reception and transmission and states.	
Particularities and Limitations	
-	
-	
Call context	
> TASK	

- > This function is Synchronous
- > This function is Non-Reentrant

Table 5-36 SoAd_MainFunction

5.3 Services used by Socket Adaptor

In the following table services provided by other components, which are used by the Socket Adaptor are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError
TcpIp	TcpIp_SoAdGetSocket
TcpIp	TcpIp_ChangeParameter
TcpIp	TcpIp_Bind
TcpIp	TcpIp_TcpListen
TcpIp	TcpIp_TcpConnect
TcpIp	TcpIp_TcpTransmit
TcpIp	TcpIp_UdpTransmit
TcpIp	TcpIp_TcpReceived
TcpIp	TcpIp_Close
TcpIp	TcpIp_RequestIpAddrAssignment
TcpIp	TcpIp_ReleaseIpAddrAssignment
TcpIp	TcpIp_GetIpAddr
TcpIp	TcpIp_GetCtrlIdx
TcpIp	TcpIp_GetRemotePhysAddr
TcpIp	TcpIp_DhcpReadOption
TcpIp	TcpIp_DhcpWriteOption
TcpIp	TcpIp_DhcpV6ReadOption
TcpIp	TcpIp_DhcpV6WriteOption
IpBase	IpBase_Copy
Linux/QNX/INTEGRITY	accept
Linux/QNX/INTEGRITY	bind
Linux/QNX/INTEGRITY	connect
Linux/QNX/INTEGRITY	fcntl
Linux/QNX/INTEGRITY	freeifaddrs
Linux/QNX/INTEGRITY	gethostname

Component	API
Linux/QNX/INTEGRITY	getifaddrs
Linux/QNX/INTEGRITY	if_nametoindex
Linux/QNX/INTEGRITY	If_indextoname
Linux/QNX/INTEGRITY	listen
Linux/QNX/INTEGRITY	select
Linux/QNX	recv
Linux/QNX	recvfrom
Linux/QNX/INTEGRITY	send
Linux/QNX/INTEGRITY	sendto
Linux/QNX/INTEGRITY	sethostname
Linux/QNX/INTEGRITY	setsockopt
Linux/QNX/INTEGRITY	close
Linux/QNX/INTEGRITY	socket
Linux/INTEGRITY	ioctl
QNX/INTEGRITY	recvmsg

Table 5-37 Services used by the Socket Adaptor

5.4 Callback Functions

This chapter describes the callback functions that are implemented by the Socket Adaptor and can be invoked by other modules. The prototypes of the callback functions are provided in the header file `SoAd_Cbk.h` by the Socket Adaptor.

5.4.1 SoAd_RxIndication

Prototype	
<pre>void SoAd_RxIndication (SoAd_SocketIdType SocketId, SoAd_SockAddrType *RemoteAddrPtr, uint8 *BufPtr, uint16 Length)</pre>	
Parameter	
SocketId [in]	Socket identifier.
RemoteAddrPtr [in]	Pointer to remote address. [Points to one of the following structs depending on value of struct element domain: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
BufPtr [in]	Pointer to buffer of received data.
Length [in]	Length of received data.

Return code	
void	none
Functional Description	
Receives data from sockets.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-38 SoAd_RxIndication

5.4.2 SoAd_CopyTxData

Prototype	
BufReq_ReturnType SoAd_CopyTxData (SoAd_SocketIdType SocketId, uint8 *BufPtr, uint16 *BufLengthPtr)	
Parameter	
SocketId [in]	Socket identifier.
BufPtr [in]	Pointer to buffer of provided transmission buffer.
BufLength BufLengthPtr [in,out]	Pointer to length of provided transmission buffer.
Return code	
BufReq_ReturnType	BUFREQ_OK Copy request accepted.
BufReq_ReturnType	BUFREQ_E_NOT_OK Copy request not accepted.
Functional Description	
Copies data to provided transmission buffer. Uses "BufLengthPtr" to update length if less data is copied to provided buffer.	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-39 SoAd_CopyTxData

5.4.3 SoAd_TxConfirmation

Prototype	
void SoAd_TxConfirmation (SoAd_SocketIdType SocketId, uint16 Length)	
Parameter	
SocketId [in]	Socket identifier.
Length [in]	Length of confirmed data.
Return code	
void	none
Functional Description	
Confirms transmission of data.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-40 SoAd_TxConfirmation

5.4.4 SoAd_LocalIpAddrAssignmentChg

Prototype	
void SoAd_LocalIpAddrAssignmentChg (SoAd_LocalAddrIdType IpAddrId, SoAd_IpAddrStateType State)	
Parameter	
IpAddrId [in]	IP address identifier.
State [in]	State of IP address assignment.
Return code	
void	none
Functional Description	
Receives local IP address assignment state changes.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-41 SoAd_LocallpAddrAssignmentChg

5.4.5 SoAd_TcpAccepted

Prototype	
Std_ReturnType SoAd_TcpAccepted (SoAd_SocketIdType SocketId, SoAd_SocketIdType SocketIdConnected, SoAd_SockAddrType *RemoteAddrPtr)	
Parameter	
SocketId [in]	Listen socket identifier.
SocketIdConnected [in]	Connected socket identifier.
RemoteAddrPtr [in]	Pointer to remote address. [Points to one of the following structs depending on value of struct element domain: - SoAd_SockAddrInetType for IPv4 - SoAd_SockAddrInet6Type for IPv6]
Return code	
Std_ReturnType	E_OK Connection was accepted.
Std_ReturnType	E_NOT_OK Connection was not accepted.
Functional Description	
Accepts TCP connections on a listen socket. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-42 SoAd_TcpAccepted

5.4.6 SoAd_TcpConnected

Prototype	
void SoAd_TcpConnected (SoAd_SocketIdType SocketId)	
Parameter	
SocketId [in]	Socket identifier.
Return code	
void	none
Functional Description	
Handles TCP connections which have been initiated locally and are now successfully connected. -	

Particularities and Limitations
-
Call context
> TASK ISR2 > This function is Synchronous > This function is Reentrant

Table 5-43 SoAd_TcpConnected

5.4.7 SoAd_TcpIpEvent

Prototype	
void SoAd_TcpIpEvent (SoAd_SocketIdType SocketId, SoAd_EventType Event)	
Parameter	
SocketId [in]	Socket identifier.
Event [in]	Event type. [TCPIP_TCP_RESET, TCPIP_TCP_CLOSED, TCPIP_TCP_FIN_RECEIVED, TCPIP_UDP_CLOSED]
Return code	
void	none
Functional Description	
Handles events on sockets. -	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-44 SoAd_TcpIpEvent

5.4.8 SoAd_DhcpEvent

Prototype	
void SoAd_DhcpEvent (SoAd_LocalAddrIdType IpAddrId, SoAd_DhcpEventType Event)	
Parameter	
IpAddrId [in]	IP address identifier.
Event [in]	Event type. [!= TCPIP_DHCP_EVENT_INVALID]
Return code	
void	none

Functional Description
Notifies user of a DHCP event. -
Particularities and Limitations
-
Call context
> TASK ISR2 > This function is Synchronous > This function is Reentrant

Table 5-45 SoAd_DhcpEvent

5.5 Configurable Interfaces

At its configurable interfaces the Socket Adaptor expects notification and callout functions which must be provided by the specific upper layer <Up> (e.g. PduR). The expected interface depends on configuration of each upper layer.

Availability, configuration dependencies and function prototypes are described in the following sub-chapters for each function.

5.5.1 <Up>_[SoAd][If]RxIndication

Prototype	
<pre>void <Up>_[SoAd][If]RxIndication (PduIdType RxPduId, const PduInfoType* PduInfoPtr)</pre>	
Parameter	
RxPduId [in]	Rx PDU identifier
PduInfoPtr [in]	Pointer to PDU
Return code	
void	none
Functional Description	
Receives IF-PDU.	
Particularities and Limitations	
-	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-46 <Up>_[SoAd][If]RxIndication

5.5.2 <Up>_[SoAd][If]TriggerTransmit

Prototype	
Std_ReturnType <Up>_[SoAd][If]TriggerTransmit (PduIdType TxPduId, PduInfoType* PduInfoPtr)	
Parameter	
TxPduId [in]	Tx PDU identifier
PduInfoPtr [in/out]	Pointer to PDU
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Copies data for a previously requested PDU via SoAd_IfTransmit if trigger transmit is used for the PDU.	
Particularities and Limitations	
- Available for upper layer with IF-API and enabled 'SoAdRoutingGroupTxTriggerable' for at least one routing group referenced in any 'SoAdPduRoute' or if 'SoAdTxIfTriggerTransmit' is enabled. PduInfoPtr->SduLength is set to provided buffer size and upper layer has to consider this value before copying to buffer.	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-47 <Up>_[SoAd][If]TriggerTransmit

5.5.3 <Up>_[SoAd][If]TxConfirmation

Prototype	
void <Up>_[SoAd][If]TxConfirmation (PduIdType TxPduId)	
Parameter	
TxPduId [in]	Tx PDU identifier
Return code	
void	none
Functional Description	
Confirms transmission of an IF-PDU.	
Particularities and Limitations	
-	

Available for upper layer with IF-API and 'SoAdIfTxConfirmation' in 'SoAdBswModules' is enabled.

Call context

- > TASK|ISR2
- > This function is Synchronous
- > This function is Reentrant

Table 5-48 <Up>_[SoAd][If]TxConfirmation

5.5.4 <Up>_[SoAd][Tp]StartOfReception

Prototype

```
BufReq_ReturnType <Up>_[SoAd][Tp]StartOfReception (PduIdType RxPduId,
PduInfoType* info, PduLengthType TpSduLength, PduLengthType* bufferSizePtr)
```

Parameter

RxPduId [in]	Rx PDU identifier
info [in]	No used [range: NULL_PTR]
TpSduLength [in]	Total length of PDU to be received
bufferSizePtr [out]	Available receive buffer

Return code

BufReq_ReturnType	BUFREQ_OK Reception request was accepted.
BufReq_ReturnType	BUFREQ_E_NOT_OK Reception request was not accepted.

Functional Description

Starts reception of a TP-PDU.

Particularities and Limitations

-

'BUFREQ_E_OVFL' as return value is treated like 'BUFREQ_E_NOT_OK'.

Parameter 'info' can be configured to constant pointer by enabling 'SoAdTpStartOfReceptionWithConstPointer'.

Call context

- > TASK|ISR2
- > This function is Reentrant

Table 5-49 <Up>_[SoAd][Tp]StartOfReception

5.5.5 <Up>_[SoAd][Tp]CopyRxData

Prototype

```
BufReq_ReturnType <Up>_[SoAd][Tp]CopyRxData (PduIdType RxPduId, const
PduInfoType* PduInfoPtr, PduLengthType* bufferSizePtr)
```

Parameter

RxPduId [in]	Rx PDU identifier
--------------	-------------------

PduInfoPtr [in]	Pointer to PDU
bufferSizePtr [out]	Available receive buffer
Return code	
BufReq_ReturnType	BUFREQ_OK Copy request was accepted.
BufReq_ReturnType	BUFREQ_E_NOT_OK Copy request was not accepted.
Functional Description	
Copies received data of a TP-PDU.	
Particularities and Limitations	
- 'BUFREQ_E_OVFL' as return value is treated like 'BUFREQ_E_NOT_OK'. Parameter 'PduInfoPtr' can be configured to constant pointer by enabling 'SoAdTpCopyRxDataWithConstPointer'.	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-50 <Up>_[SoAd][Tp]CopyRxData

5.5.6 <Up>_[SoAd][Tp]RxIndication

Prototype	
void <Up>_[SoAd][Tp]RxIndication (PduIdType RxPduId, Std_ReturnType result)	
Parameter	
RxPduId [in]	Rx PDU identifier
result [in]	Reception result
Return code	
void	none
Functional Description	
Indicates that a TP-PDU reception is finished.	
Particularities and Limitations	
- -	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-51 <Up>_[SoAd][Tp]RxIndication

5.5.7 <Up>_[SoAd][Tp]CopyTxData

Prototype	
BufReq_ReturnType <Up>_[SoAd][Tp]CopyTxData (PduIdType TxPduId, const PduInfoType* PduInfoPtr, RetryInfoType* retry, PduLengthType* availableDataPtr)	
Parameter	
TxPduId [in]	Tx PDU identifier
PduInfoPtr [in/out]	Pointer to PDU
retry [in]	Not used [range: NULL_PTR]
availableDataPtr [out]	Available transmission buffer
Return code	
BufReq_ReturnType	BUFREQ_OK Copy request was accepted.
BufReq_ReturnType	BUFREQ_E_NOT_OK Copy request was not accepted.
Functional Description	
Copies data to be transmitted of a TP-PDU.	
Particularities and Limitations	
- 'BUFREQ_E_OVFL' as return value is treated like 'BUFREQ_E_NOT_OK'. Parameter 'PduInfoPtr' can be configured to constant pointer by enabling 'SoAdTpCopyTxDataWithConstPointer'.	
Call context	
> TASK ISR2 > This function is Reentrant	

Table 5-52 <Up>_[SoAd][Tp]CopyTxData

5.5.8 <Up>_[SoAd][Tp]TxConfirmation

Prototype	
void <Up>_[SoAd][Tp]TxConfirmation (PduIdType TxPduId, Std_ReturnType result)	
Parameter	
TxPduId [in]	Tx PDU identifier
result [in]	Transmission result
Return code	
void	none
Functional Description	
Confirms transmission of a TP-PDU.	
Particularities and Limitations	
- -	

Call context
> TASK ISR2
> This function is Reentrant

Table 5-53 <Up>_SoAd][Tp]TxConfirmation

5.5.9 <Up>_SoConModeChg

Prototype	
void <Up>_SoConModeChg (SoAd_SoConIdType SoConId, SoAd_SoConModeType Mode)	
Parameter	
SoConId [in]	Socket connection identifier
Mode [in]	New socket connection mode
Return code	
void	None
Functional Description	
Notifies about a socket connection mode change	
Particularities and Limitations	
- Available if 'SoAdBswModules/SoAdSoConModeChg' is enabled for an upper layer or if a 'SoAdGeneral/SoAdAdditionalSoConModeChgCallback' is configured.	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-54 <Up>_SoConModeChg

5.5.10 <Up>_LocalIpAddrAssignmentChg

Prototype	
void <Up>_LocalIpAddrAssignmentChg (SoAd_SoConIdType SoConId, SoAd_IpAddrStateType State)	
Parameter	
SoConId [in]	Socket connection identifier
State [in]	New socket connection local IP address state
Return code	
void	None
Functional Description	
Notifies about IP assignment state for a specific socket connection.	

Return code	
void	None
Functional Description	
Notifies about created TCP TLS socket.	
Particularities and Limitations	
-	
Available if 'SoAdTcpTlsCallback/SoAdTcpTlsSocketCreatedNotification' is configured.	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-57 <Up_TcpTlsSocketCreatedNotification>

5.5.13 <Up_VerifyRxPdu>

Prototype	
Std_ReturnType <Up_VerifyRxPdu> (const SoAd_SockAddrType * LocalAddrPtr, const SoAd_SockAddrType * RemoteAddrPtr, SoAd_PduHdrIdType PduHdrId, const PduInfoType * PduInfoPtr)	
Parameter	
LocalAddrPtr [in]	Pointer to local socket address
RemoteAddrPtr [in]	Pointer to remote socket address
PduHdrId [in]	PDU Header ID
PduInfoPtr [in]	Pointer to PDU data [range: NULL_PTR if SoAdVerifyRxPduMaxDataLength == 0]
Return code	
Std_ReturnType	E_OK PDU reception verification succeeded
Std_ReturnType	E_NOT_OK PDU reception verification failed
Functional Description	
Verifies a PDU reception and indicates if a reception shall be continued or dropped.	
Particularities and Limitations	
-	
Configurable in 'SoAdVerifyRxPduCallback/SoAdVerifyRxPdu'	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-58 <Up_VerifyRxPdu>

5.5.14 <Up_DhcpEvent>

Prototype	
void <Up_DhcpEvent> (SoAd_LocalAddrIdType IpAddrId, SoAd_DhcpEventType Event)	
Parameter	
IpAddrId [in]	IP address identifier.
Event [in]	Event type. [!= TCPIP_DHCP_EVENT_INVALID]
Return code	
void	none
Functional Description	
Notifies user of a DHCP event.	
-	
Particularities and Limitations	
-	
Call context	
> TASK ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-59 <Up_DhcpEvent>

6 Configuration

There is one configuration tool to configure and generate the Socket Adaptor:

- > DaVinci Configurator Pro

6.1 Configuration Variants

The Socket Adaptor supports the configuration variants

- > VARIANT-PRE-COMPILE
- > VARIANT-POST-BUILD-LOADABLE

The configuration classes of the Socket Adaptor parameters depend on the supported configuration variants. For their definitions please see the SoAd_bswmd.arxml file.

6.2 Configuration with DaVinci Configurator Pro

The Socket Adaptor is configured with the help of the configuration tool DaVinci Configurator Pro.

In the following sub-chapters some configuration hints are given to understand how to configure the Socket Adaptor correctly.

6.2.1 Socket Connection handling

6.2.1.1 Socket Connection Group

To support multiple communication partners on one local socket (e.g. multiple clients on one server) the Socket Adaptor can define multiple socket connections. The general difference between these socket connections is the remote address (address of communication partner). All common properties are defined in socket connection group configuration (refer to Figure 6-1 and Figure 6-2).

Socket Connection Group

Socket Connection Group properties

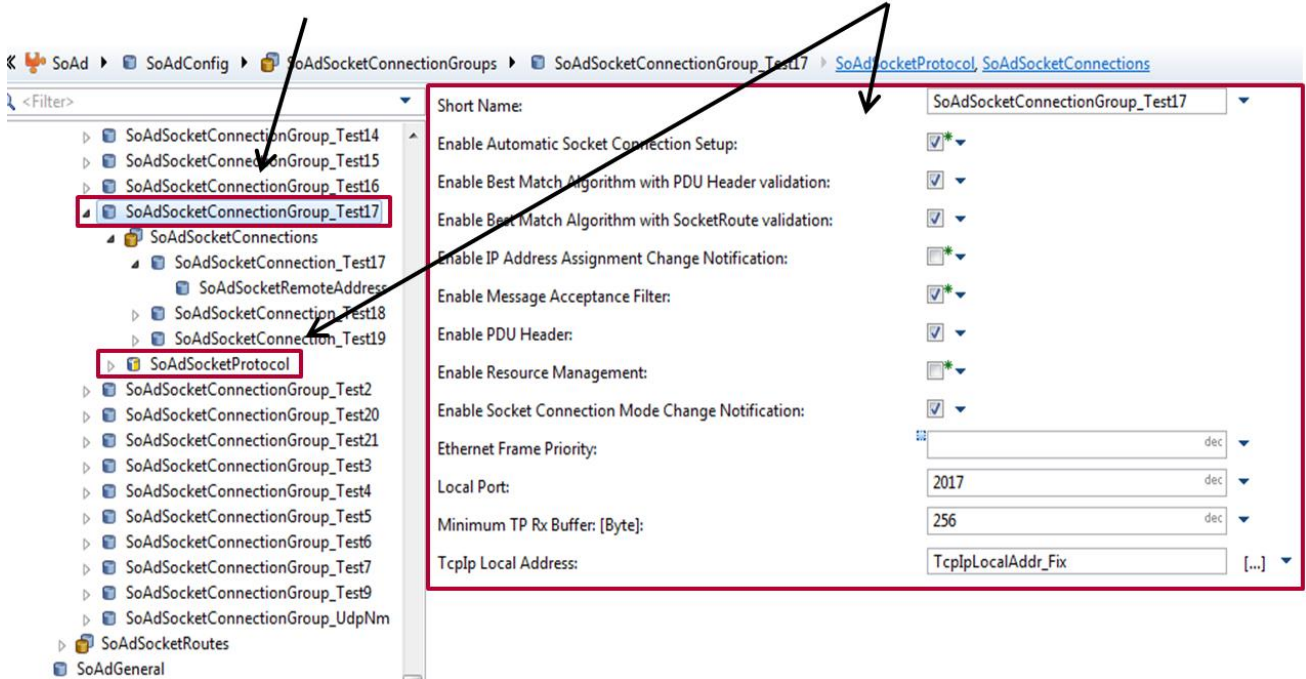


Figure 6-1 Socket Connection Group configuration

Socket Connection

Socket Connection properties

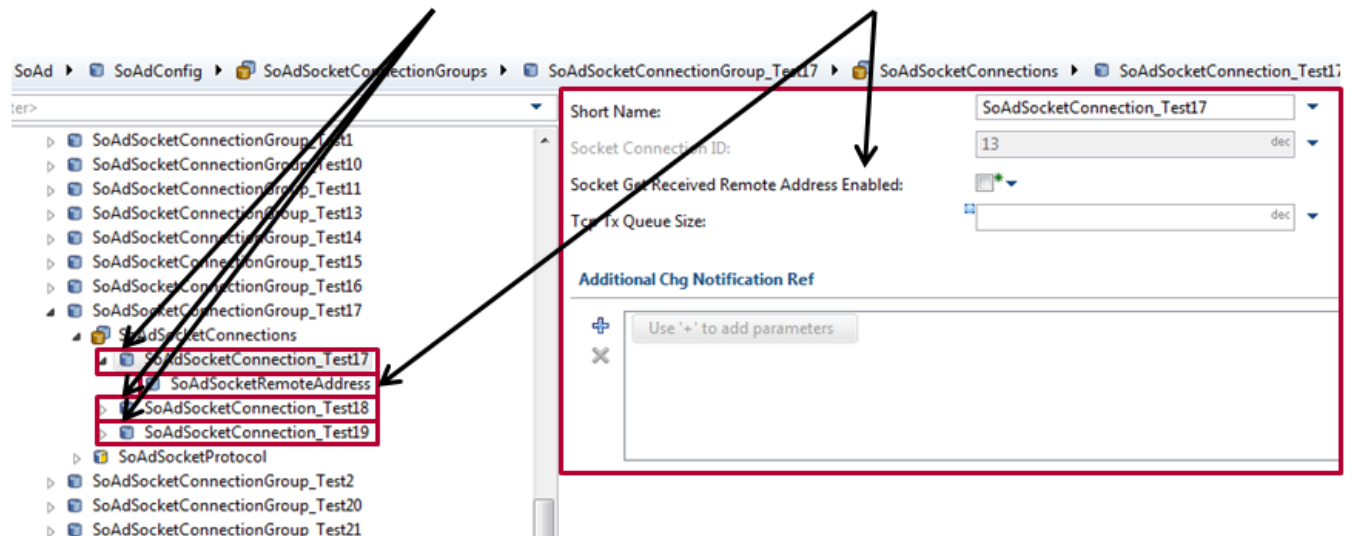


Figure 6-2 Socket Connection configuration

In case of TCP and special cases of UDP buffers must be configured in the TcpIp module. The tool provides solving actions to add and optimize buffers.

6.2.1.2 Socket Connection establishment

There are basically two different ways how to handle socket connections. Depending on the use case following ways can be chosen:

- > Manual
- > Automatic

If an upper layer needs to control the socket connection state or remote address a manual socket connection establishment is recommended (e.g. DoIP). In all other cases Socket Adaptor can handle the socket connection on itself (e.g. PduR).

6.2.1.2.1 Manual

Manual socket connection establishment is enabled if the corresponding parameter is disabled (Figure 6-3).

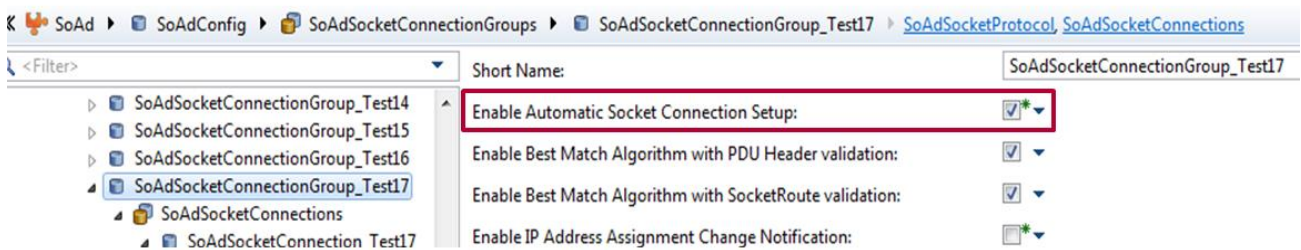


Figure 6-3 Socket Connection Setup parameter

Following services are now available:

- > SoAd_OpenSoCon()
- > SoAd_CloseSoCon()
- > SoAd_SetRemoteAddr()
- > SoAd_SetUniqueRemoteAddr()

These services can be used to open and close a socket connection or set the remote address.

In case of UDP, the socket connection can be opened on reception if the message acceptance filter is enabled in the socket connection group and the remote address contains wildcards.

To set a port to wildcard, set port to 0. To configure an IP address to wildcard, empty the remote IP address field. Vector supports a special value for a not set IP address or port (Figure 6-4). If an IP address or port is not set, a socket connection is not considered in the best match algorithm (i.e. socket connection won't be opened on reception).

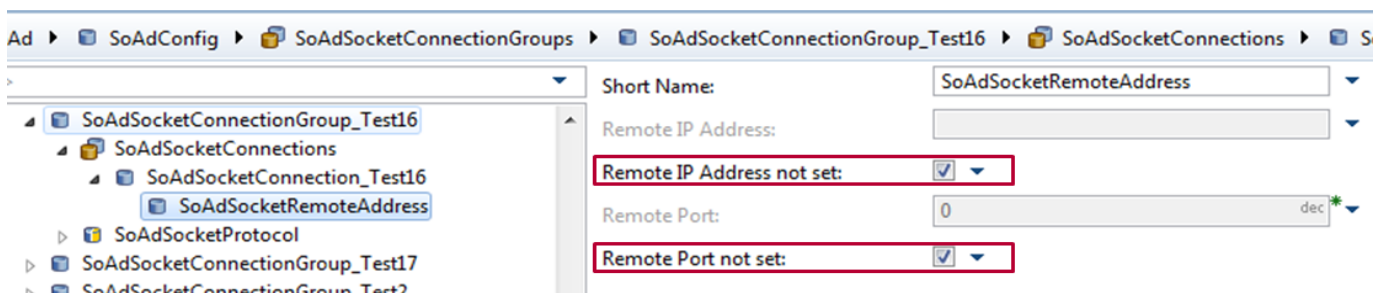


Figure 6-4 Remote address "not set" parameter

6.2.1.2.2 Automatic

Automatic socket connection establishment is enabled if the corresponding parameter is enabled (Figure 6-3).

All services described in chapter 6.2.1.2.1 are not available and corresponding functions will return E_NOT_OK on call.

A socket connection is automatically opened if the remote address is set in configuration or on reception in case of UDP and the message acceptance filter is enabled in socket connection group.

6.2.2 Transmission path

This chapter gives a short description about how configure a transmission path.

1. Configure a socket connection according to the use case (chapter 6.2.1)
2. Add a PDU Route and choose referenced PDU and upper layer API type (Figure 6-5)

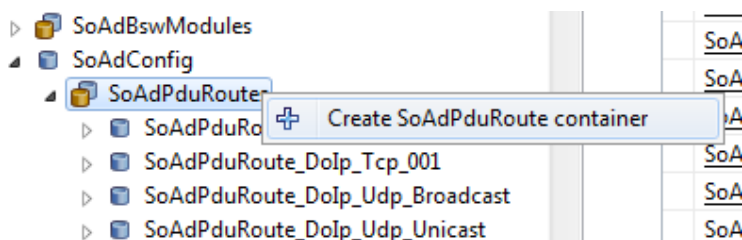


Figure 6-5 Add PDU Route

3. Add one or multiple (fan-out) PDU Route Destination (Figure 6-6) and reference the socket connection created in 1

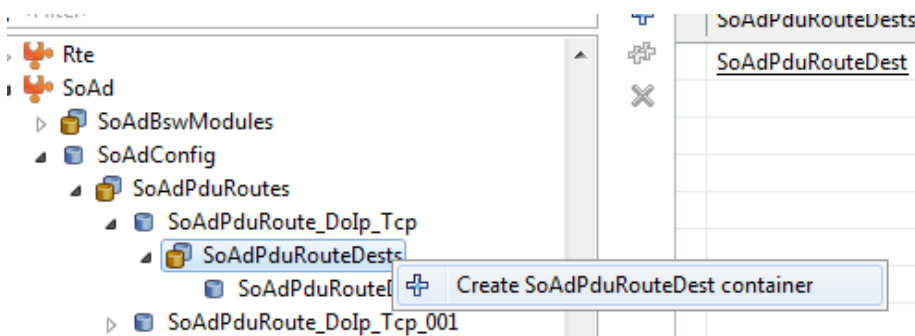


Figure 6-6 Add PDU Route Destination

6.2.3 Reception path

1. Configure a socket connection according to the use case (chapter 6.2.1)
2. Add a Socket Route and reference socket connection created in 1(Figure 6-7)

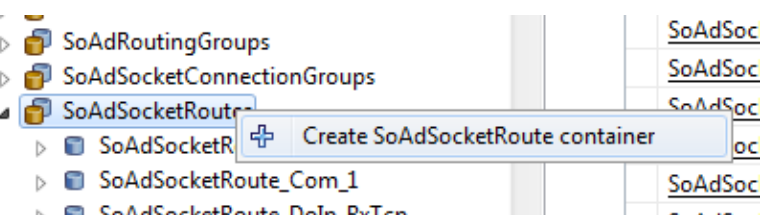


Figure 6-7 Add Socket Route

3. Add exactly one (fan-out not possible) Socket Route Destination and chose the upper layer PDU(Figure 6-8)

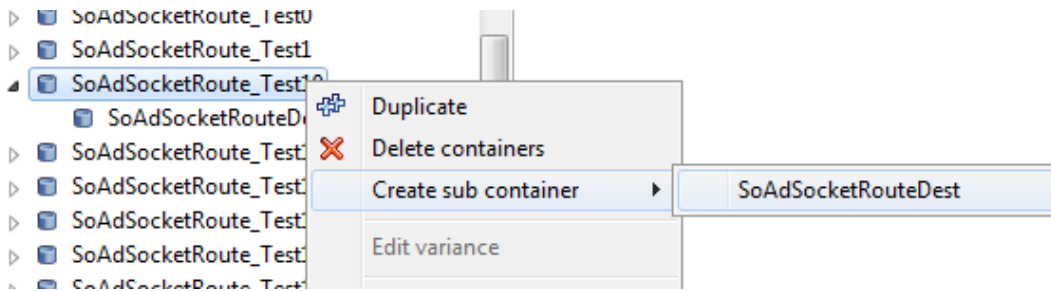


Figure 6-8 Add Socket Route Destination

6.2.4 Vector specific feature

6.2.4.1 Best Match Algorithm extensions

Best Match Algorithm according to [1] chooses a socket connection of a socket connection group considering remote addresses. In some cases it may be useful to consider more than remote address. Vector implements extensions of the Best Match Algorithm described here:

> Best Match Algorithm with PDU Header validation

If this option is enabled PDU Header is extracted before socket connection is chosen. After PDU Header is received completely Best Match Algorithm is used to find the suitable socket connection. If the socket connection is found an additional check is performed to verify that the received PDU Header ID matches the configured value. In case of no match Best Match Algorithm will continue. If no socket connection and no socket route could be found the received PDU will be discarded.

> Best Match Algorithm with Socket Route validation

This extension considers the existence of a socket route at a socket connection. If no socket route is configured for a socket connection the Best Match Algorithm will continue. This option can be used to prevent a socket connection setup on reception of data while other reception socket connections are used in same socket connection group.

Both described feature can be configured in the socket connection group (refer to Figure 6-9).

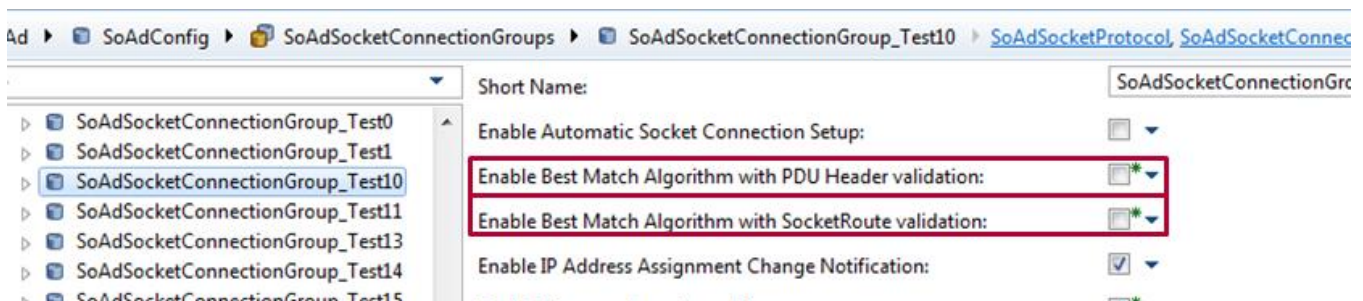


Figure 6-9 Best Match Algorithm extensions

6.2.4.2 UDP Immediate IF TxConfirmation

This feature is used to confirm data within interrupt level after successfully sending data. How to configure this feature is described in Figure 6-10.

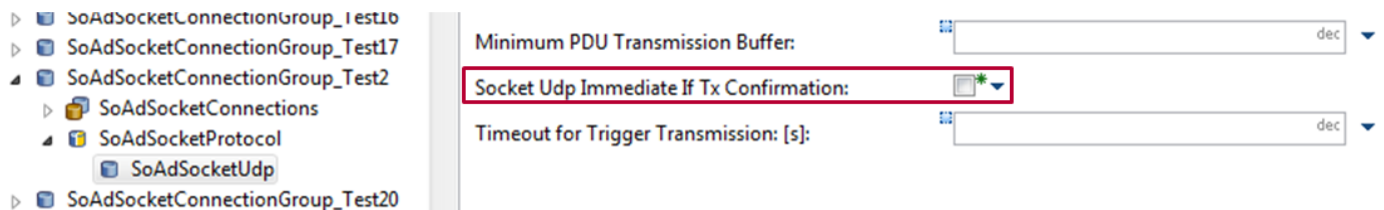


Figure 6-10 UDP Immediate IF TxConfirmation

According to [1] TxConfirmation for UDP socket connections with IF API is called within main function. It may be possible that new data are received while sent data are not confirmed yet. This may lead to incorrect behavior if upper layer is based on request and response behavior. Immediate TxConfirmation can prevent reception before confirmation.

This feature is implemented for one PduRoute/PduRouteDestination per socket connection only.

6.2.4.3 SoAd_GetRcvRemoteAddr()

Via this API it is possible for the upper layer to retrieve the remote address of the last received message on a socket connection. To enable this feature enable the parameter marked in Figure 6-11.

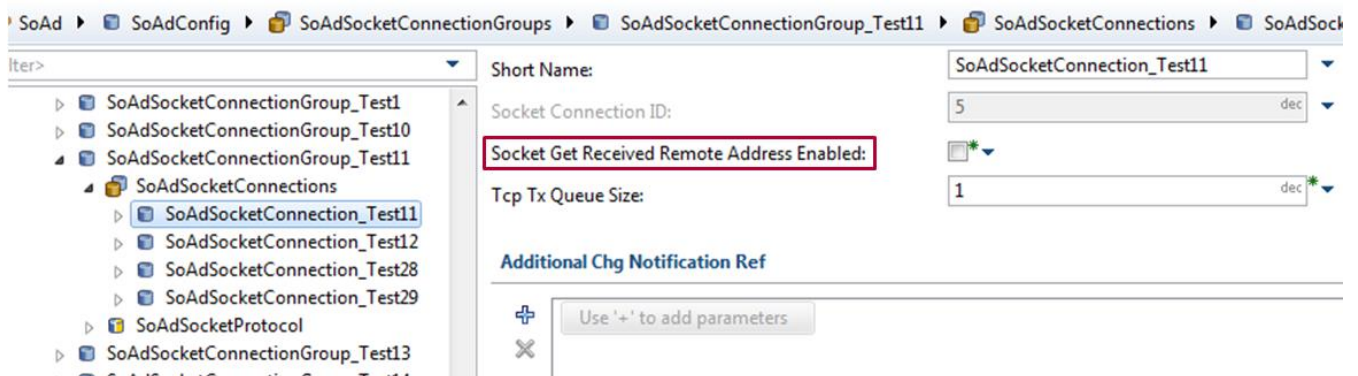


Figure 6-11 SoAd_GetRcvRemoteAddr

6.2.4.4 Additional SoConModeChg notification

It is possible for Socket Adaptor upper layers to get a socket connection mode change notification on a socket connection which is not referenced by a SoAdPduRoute or SoAdSocketRouteDestination to this upper layer.

To configure this additional notification add a reference to corresponding SoAdBswModule as described in Figure 6-12 on the SoAdSocketConnection.

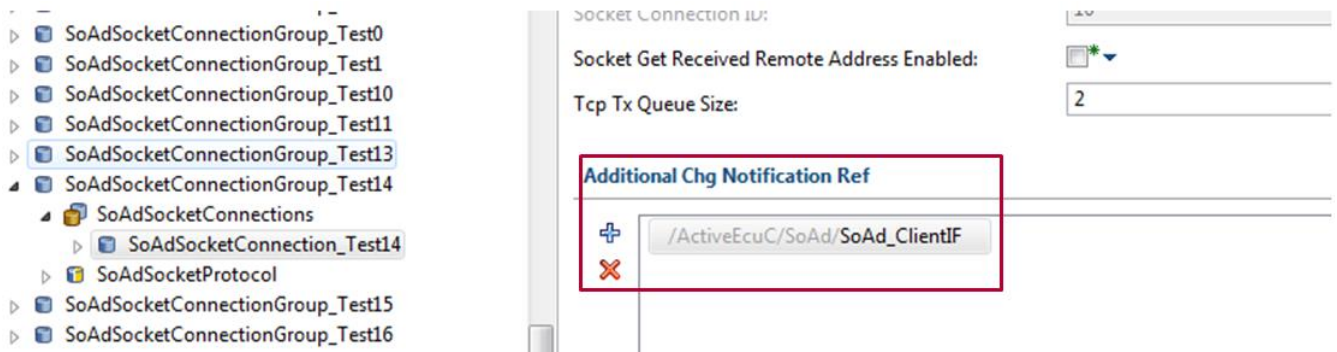


Figure 6-12 Additional SoConModeChg notification

6.2.4.5 BSD Socket API

If this option is part of the delivery Socket Adaptor can be used with BSD Socket API of Linux, QNX or INTEGRITY. In case feature is enabled AUTOSAR TcpIp stack cannot be used anymore.

The services used by Socket Adaptor in case of BSD Socket API are listed in Table 5-37.

In order to configure the Socket Adaptor for BSD Socket API a “dummy” TcpIp module has to be configured (i.e. has different description file which is activated as described in chapter 6.2.4.5.1). The TcpIp module is used to configure address relevant information used by Socket Adaptor to bind sockets on BSD Socket API.

6.2.4.5.1 Enable/Disable BSD Socket API

To enable this feature following steps are required. It depends on the delivery whether the following files are available. If a file is not available modification can be skipped.

1. Rename “TcpIp_bswmd.arxml” “TcpIp_bswmd.arxml_”.
The file can be found in “...\external\BSWMD\TcpIp”
2. Rename “Tp_AsrTpTcpIp.jar” “Tp_AsrTpTcpIp.jar_”,
rename “Tp_AsrTpTcpIp_DhcpV4Server.jar” “Tp_AsrTpTcpIp_DhcpV4Server.jar_”,
rename “Tp_AsrTpTcpIp_IpV4.jar” “Tp_AsrTpTcpIp_IpV4.jar_” and
rename “Tp_AsrTpTcpIp_IpV6.jar” “Tp_AsrTpTcpIp_IpV6.jar_”
These files can be found in “...\external\DaVinciConfigurator\Generators\TcpIp”
3. Rename “TcpIp_BSD_bswmd.arxml_” “TcpIp_BSD_bswmd.arxml”
The file can be found in “...\external\BSWMD\SoAd”
4. After starting DaVinci Configurator Pro the feature is enabled automatically by the tool.
Please remove (save configuration by export before) all Ethernet modules in your configuration (“Eth”, “EthTrcv”, “EthIf”, “EthSM”)

To disable the feature again revert the steps mentioned above (e.g. import exported Ethernet modules).

6.2.4.5.2 Configure Socket API type

Besides enabling or disabling the BSD Socket API as described above the corresponding Socket API type has to be chosen as described in Figure 6-13.

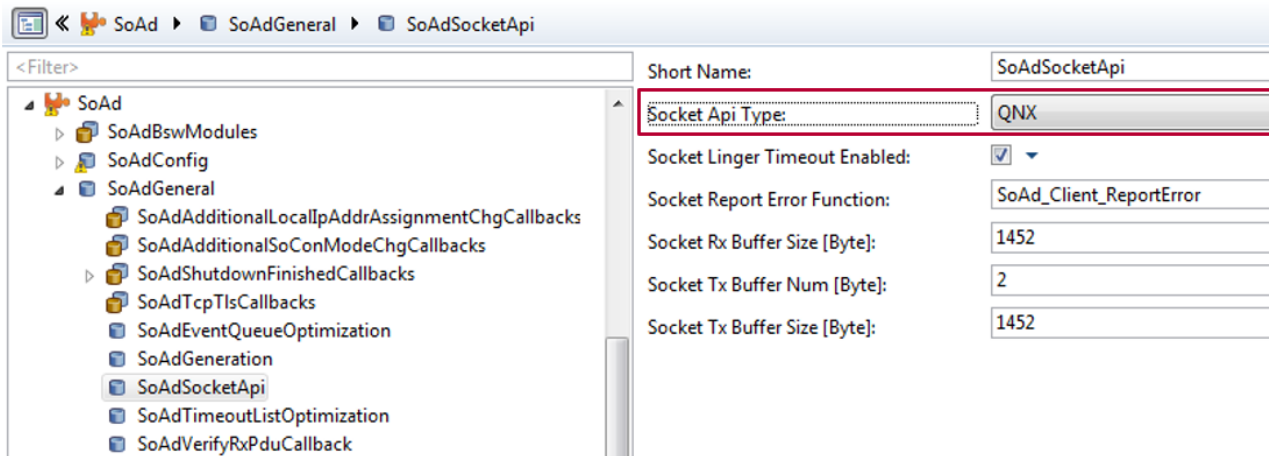


Figure 6-13 Socket API type configuration

6.2.4.5.3 Configure header file for type “sockaddr_ll”

Depending on the BSD Socket API distribution, type `sockaddr_ll` may not be defined within the already included header files. In this case add the corresponding header file via configuration as described in Figure 6-14.

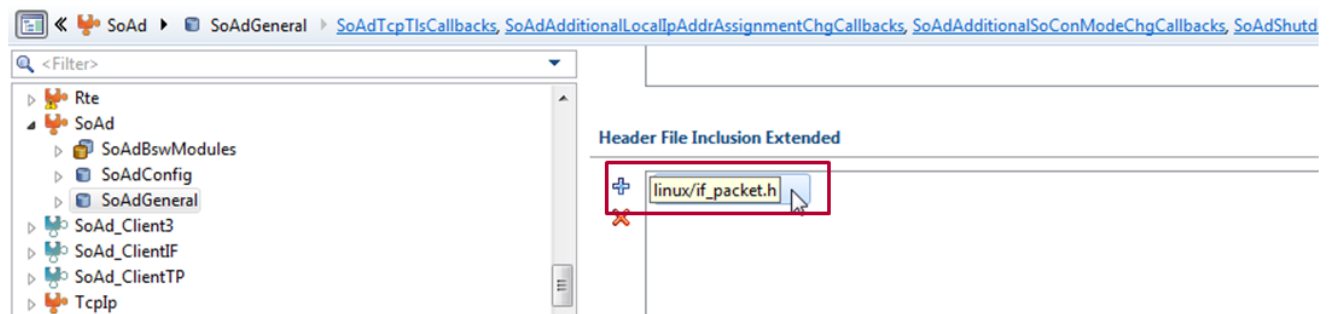


Figure 6-14 Configure header file for type “sockaddr_ll”

6.2.4.5.4 Configure local IP address

Each local IP address belongs to a network interface. Each network interface can have multiple local IP addresses. The network interfaces and their names are configured in a configuration container as described in Figure 6-15. Parameter “Ctrl Name” has to match the name configured on the interface in Linux/QNX/INTEGRITY.

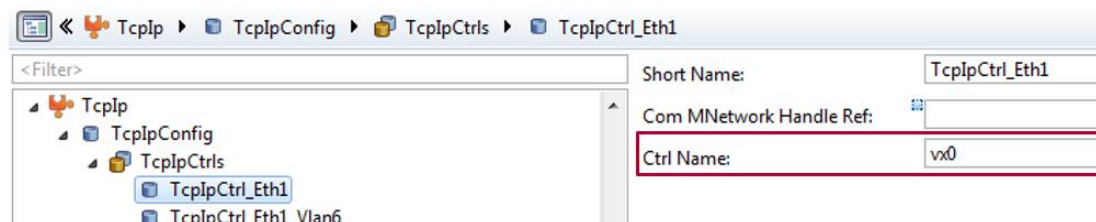


Figure 6-15 Controller/Interface configuration

For a local address it has to be decided if it is used for (Figure 6-16):

1. Unicast (transmission and reception)
2. Multicast (reception only)

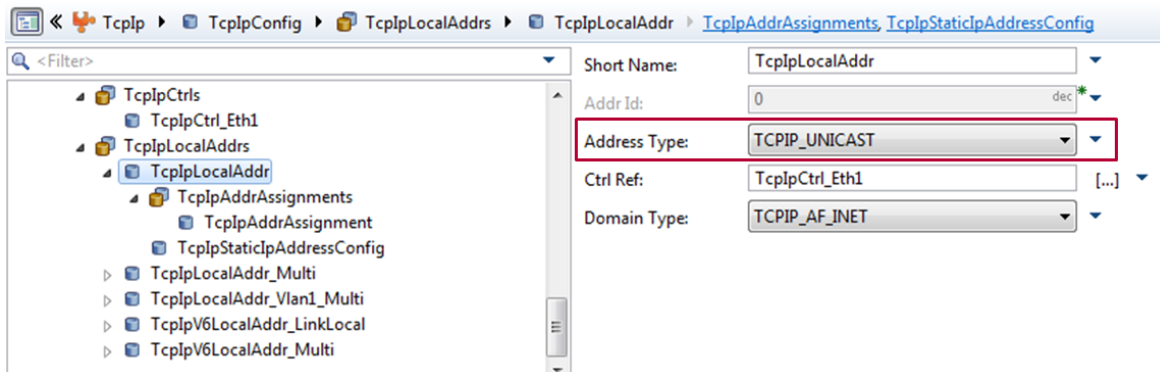


Figure 6-16 Address Type configuration

Afterwards it has to be decided which assignment method shall be used (Figure 6-17):

1. Static
2. DHCP

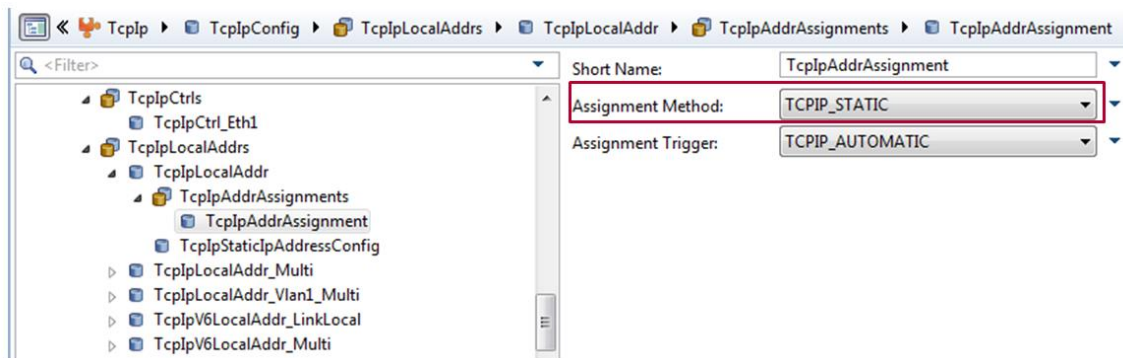


Figure 6-17 Assignment Method configuration

“Static” means that Socket Adaptor will bind corresponding sockets to this address. If the IP address is not available on the configured interface no communication is possible and Socket Adaptor will try to bind the socket in next main function again until IP address is available or IP address request has been released.

In case of “DHCP” Socket Adaptor will bind all affected sockets to the first IP address of the configured interface (except loopback address). This assignment method is valid for unicast addresses only.

For each local address an assignment trigger can be chosen (Figure 6-18):

1. Automatic
2. Manual

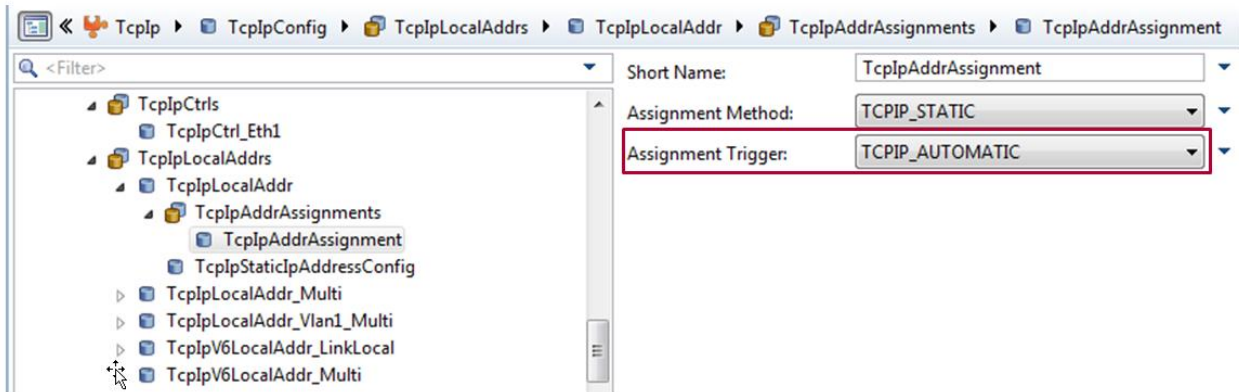


Figure 6-18 Assignment Trigger configuration

“Automatic” means that Socket Adaptor tries to assign the IP address immediately after startup (e.g. take the first IP address of interface in case of DHCP). If IP address gets lost, Socket Adaptor will try to assign this IP address again in each main function cycle until IP address is available again. In case of “DHCP” this behavior is used to handle a change of the IP address assigned by DHCP server.

For “manual” local addresses a call to `SoAd_RequestIpAddrAssignment()` is required to trigger assignment of IP address at runtime.

6.2.4.5.5 Request and release IP address

`SoAd_RequestIpAddrAssignment()` and `SoAd_ReleaseIpAddrAssignment()` do not support all cases which are described in AUTOSAR. As described in chapter before “static” and “DHCP” are supported only. For all other assignment methods `E_NOT_OK` is returned.

6.2.4.5.6 VLAN priority

For Linux Socket Adaptor supports to set the frame priority on sockets. The Socket Adaptor does not support this for other BSD Socket API distributions.

As defined for the AUTOSAR TcpIp stack the frame priority can be set as described in Figure 6-19.

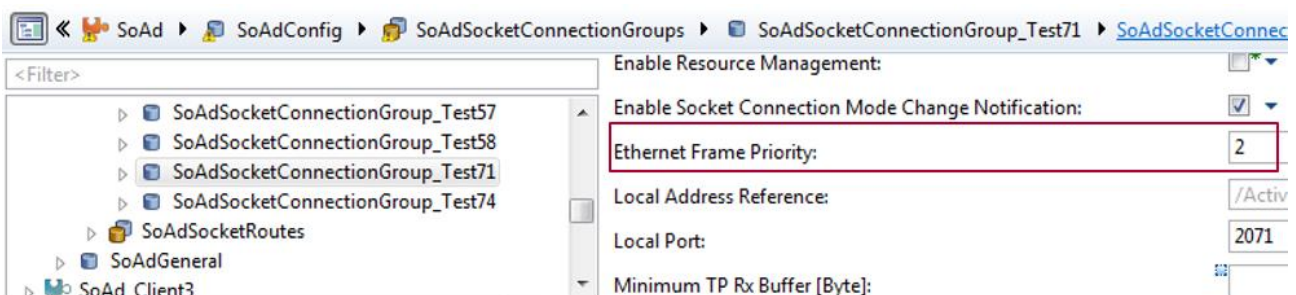


Figure 6-19 Frame Priority configuration

To set the frame priority may be not sufficient since the frame priority set on a socket is handled in the queues of the Linux stack internally. To add the VLAN priority to the frame on the bus the corresponding egress map on the interface must be set.

This can be done manually (e.g. via script running at startup) or via the Socket Adaptor.

To do this manually you may use “vconfig” if available. Refer to the example below which sets the egress map so that a socket priority set to “7” is sent with VLAN priority “7”:

```
vconfig set_egress_map eth1.6 7 7
```

If a console is available you may use the “cat” command to display the ingress/egress map:

```
cat /proc/net/vlan/eth1.6
```

To let the Socket Adaptor set the egress map enable the parameter described in Figure 6-20.

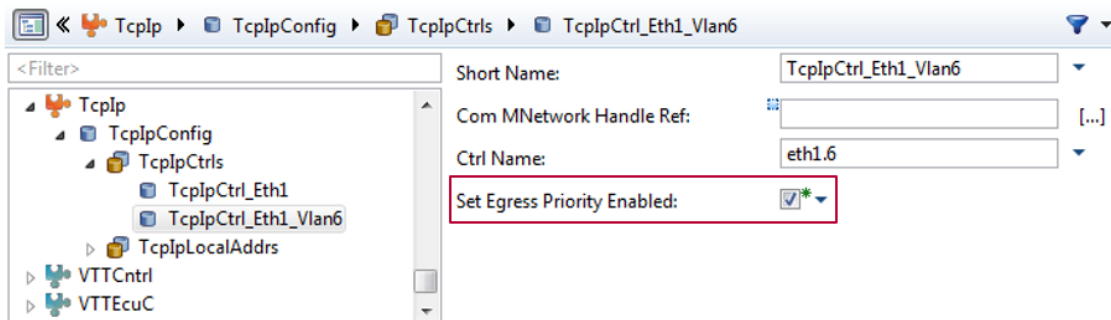


Figure 6-20 Set Egress Priority configuration

6.2.4.5.7 Broadcast/Multicast Reception

For Linux and QNX UDP Sockets, it is possible to enable or disable the reception of broadcasts. The broadcast reception can be set as shown in Figure 6-21.

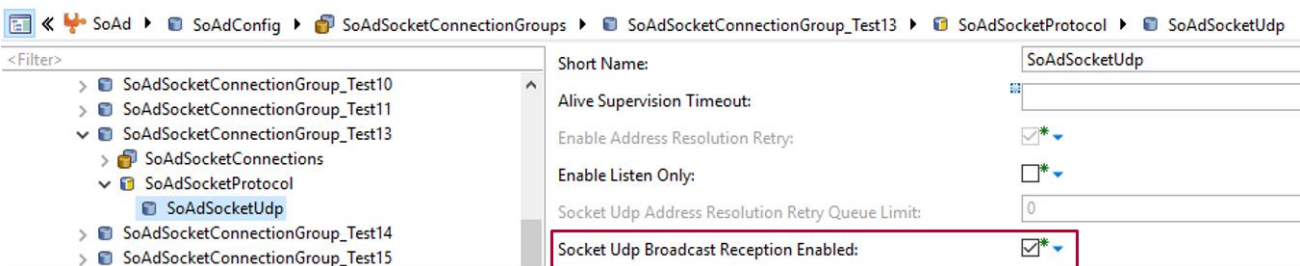


Figure 6-21 Set Broadcast Reception configuration

The option to disable broadcast reception can be used when no broadcasts (IPv4 limited and directed broadcast) or multicasts (IPv6 all nodes and all routers multicast) are required on the selected socket connection group. This can be used as an optimization to reduce runtime and resources, as no additional sockets for broadcast/multicast are bound.

The parameter must exist if the Socket API type is LINUX or QNX and if the local address references a unicast address. Otherwise, this parameter must not exist. By default, the reception of broadcasts is enabled.

6.2.4.6 Shutdown mechanism

Service `SoAd_Shutdown()` initiates a shutdown of Socket Adaptor module. Pending transmissions will be continued, transmission and reception will be disabled and Socket Adaptor is set into a shutdown state afterwards. A call to `SoAd_Init()` is required to reinitialize Socket Adaptor again.

This feature is enabled always and can be used to shutdown Socket Adaptor orderly (e.g. in case of flashing the ECU).

It is possible to configure an optional callback which is called when shutdown is finished. This callback can be configured as described in Figure 6-22 for a specific upper layer or in Figure 6-23 for any other module (please consider Figure 6-24 to include additional header files).

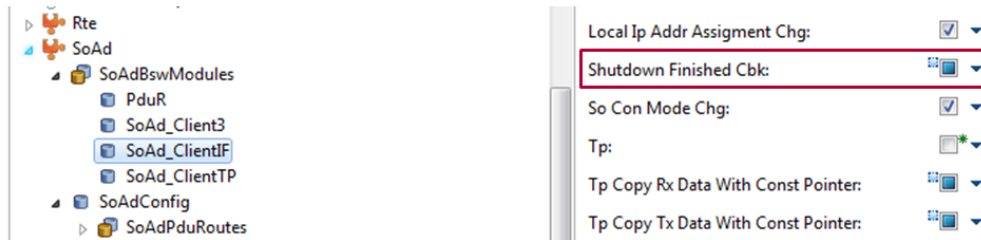


Figure 6-22 Enable upper layer specific <User>_ShutdownFinished()

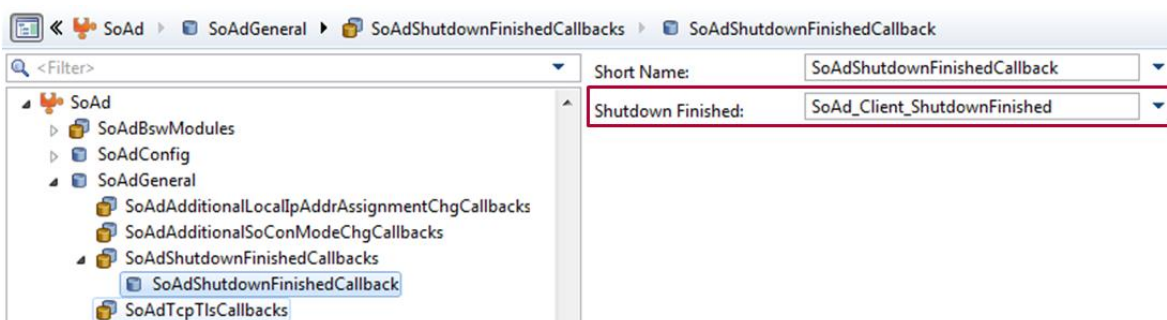


Figure 6-23 Configure <User>_ShutdownFinished() for any module

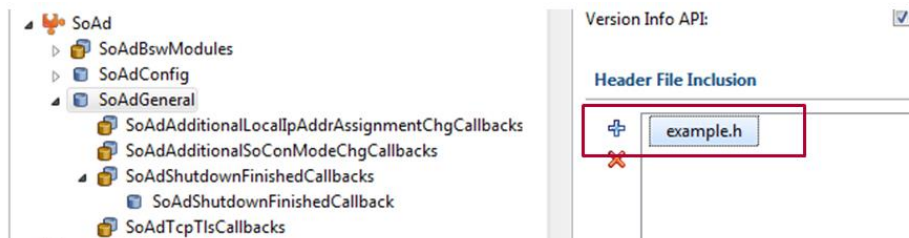


Figure 6-24 Additional header file inclusion

It is also possible to poll module state via multiple calls to `SoAd_Shutdown()`.

If pending transmission cannot be finished or sockets cannot be closed since tester does not acknowledge closing (i.e. no "FIN" flag sent) a timeout is configured to shutdown module immediately after timeout (Figure 6-25) expired.

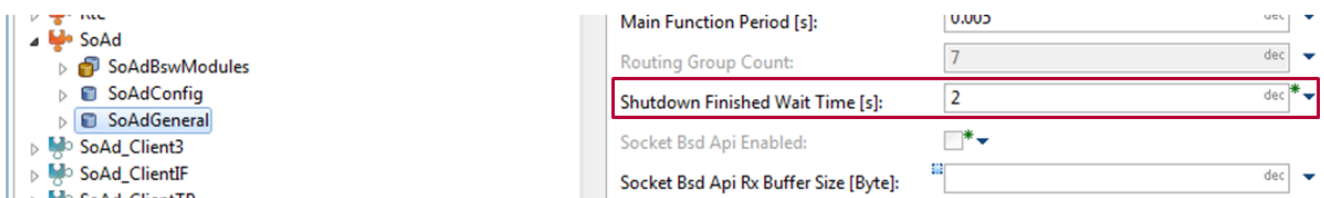


Figure 6-25 Shutdown finished timeout

6.2.4.7 TLS client

In combination with Vector TcpIp and Tls a TLS client can be configured on each TCP client socket connection (parameter `SoAdSocketTcp/SoAdTcpSocketTcpInitate` set to true).



Note

The TLS client is not supported if BSD Socket API is used.

To enable TLS client on a socket connection, sub container `SoAdTcpTlsConfig` has to be created (Figure 6-26).

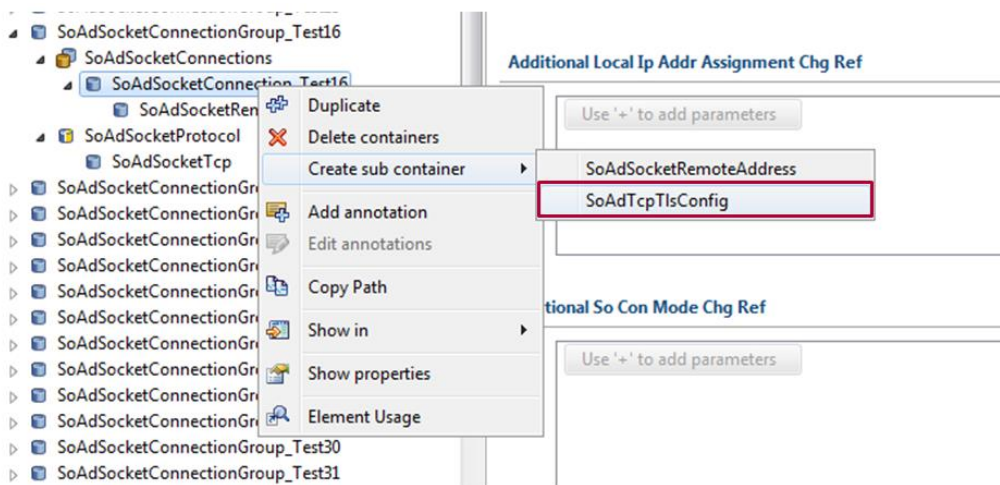


Figure 6-26 Create TLS client configuration in socket connection

Tls buffer sizes can be modified with parameters `SoAdTcpTlsTxBufferSize` and `SoAdTcpTlsRxBufferSize` (Figure 6-27). These values are validated against configuration in Tls module. Tls calculates buffers for TcpIp module and Socket Adaptor has to adapt parameters `SoAdSocketTcpTxBufferMin` and `SoAdSocketTpRxBufferMin` to match buffer sizes calculated by Tls (in a valid configuration tool provides solving actions to adapt parameters).

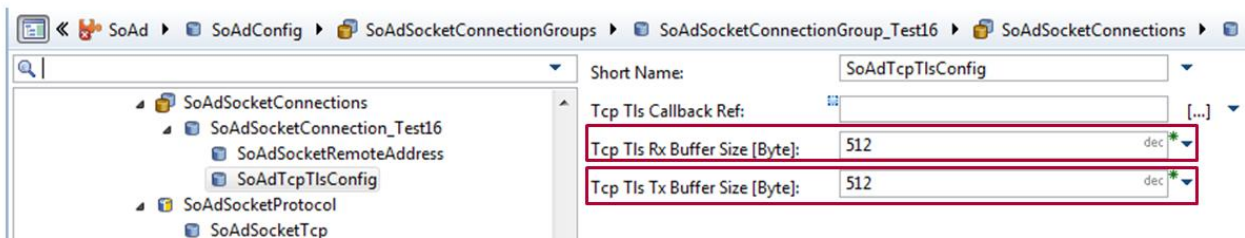


Figure 6-27 SoAd Tls Tx and Rx buffer configuration

Additionally a callback can be configured to notify a user about a socket creation in TcpIp module. Within this callback user can set additional parameters of socket (e.g. client

certificate). These parameters are not part of Socket Adaptor therefore they are not described here.

To configure a callback create a callback container (Figure 6-28) and reference this callback container or an already existing in the corresponding socket connection (Figure 6-29). If callback prototype is declared in a separate header file add it to configuration as described in Figure 6-24.

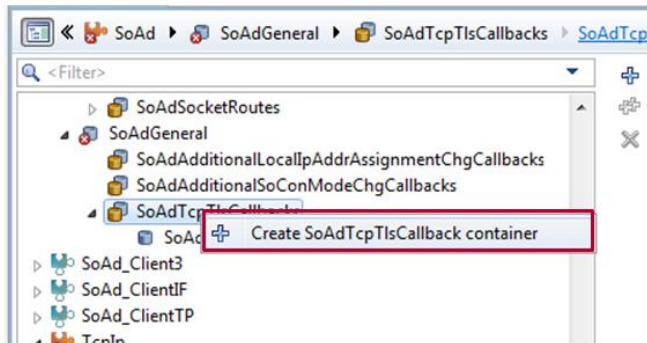


Figure 6-28 TLS callback container configuration

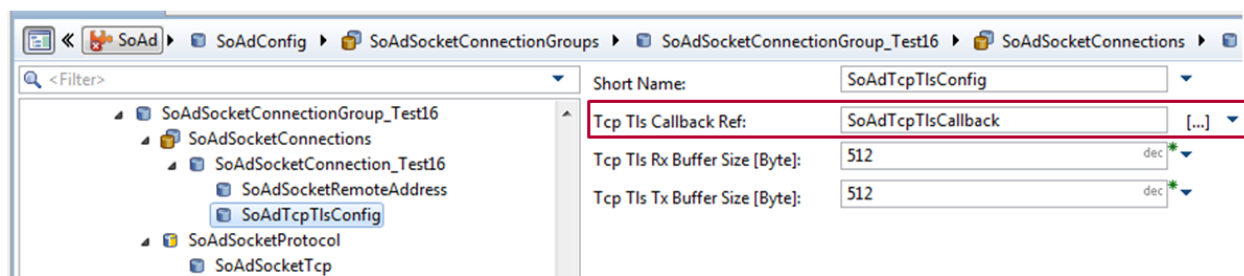


Figure 6-29 TLS callback container reference configuration

6.2.4.8 MainFunction splitting

Depending on upper layer API (TP/IF) and protocol (TCP/UDP) transmission and reception are handled in MainFunction context of Socket Adaptor (refer to [1]). Also other modules of the communication stack use MainFunctions for transmission and reception (e.g. TcpIp). If there is only one MainFunction per module it has to be decided if order of calls to the MainFunctions is optimized for transmission or reception path.

Order for reception optimization:

```
TcpIp_MainFunction() -> SoAd_MainFunction()
```

Order for transmission optimization:

```
SoAd_MainFunction() -> TcpIp_MainFunction()
```

Vector supports a spitting into Rx MainFunction and Tx MainFunction to optimize transmission and reception path parallel.

Order in combination with splitting of MainFunction of TcpIp module:

```
TcpIp_MainFunctionRx() -> SoAd_MainfunctionRx() ->
SoAd_MainfunctionTx() -> TcpIp_MainFunctionTx()
```

Additionally `SoAd_MainFunctionState()` is available to handle the module states.

To enable the feature set the following parameter to true:

`SoAd/SoAdGeneral/SoAdMainFunctionSplitEnabled`



Caution

If splitting is enabled `SoAd_MainFunction()` must not be called anymore.

6.2.4.9 Optimized TP transmission

Socket Adaptor according to AUTOSAR can handle maximum one TP transmission per MainFunction on a PDU. But some upper layers need to send data more often.

Therefore an optimized TP transmission is supported. If this feature is enabled the entire TP transmission can be handled in transmission context as described in Figure 6-30.

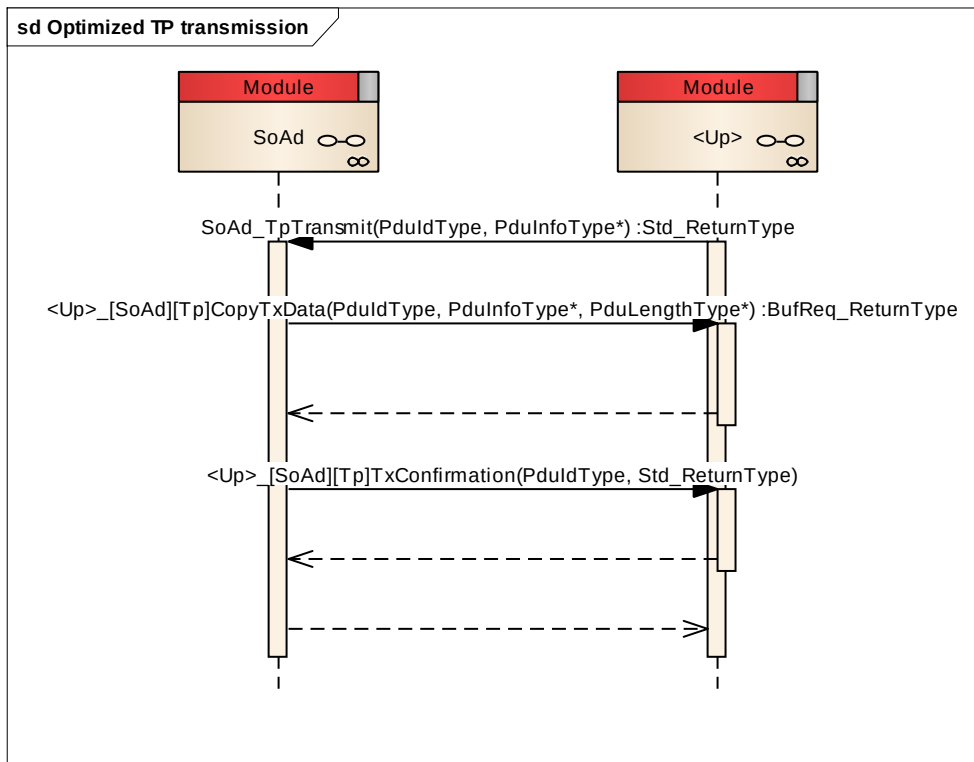


Figure 6-30 Optimized TP transmission sequence

This feature can be enabled and disabled for each TP PduRoute separately (Figure 6-31).

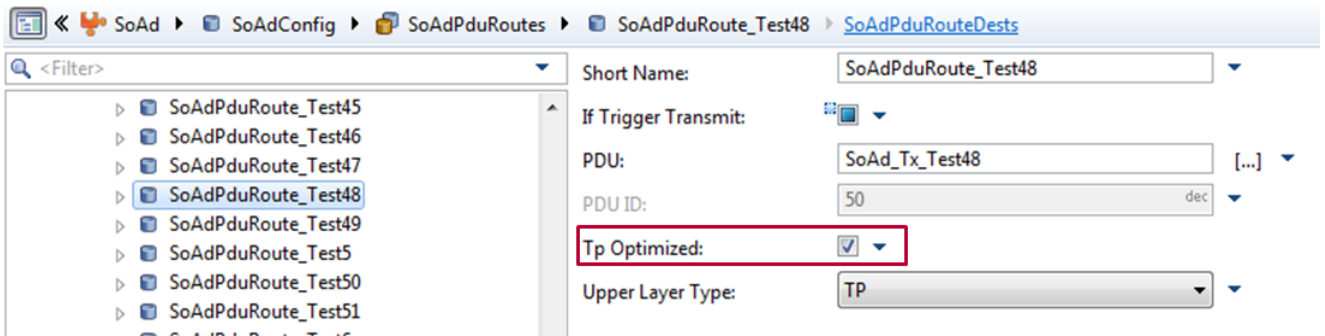


Figure 6-31 Optimized TP transmission configuration



Note

It is highly recommended to enable `SoAdSocketTcpImmediateTpTxConfirmation` if feature is used on TCP sockets since TP transmission is finished on reception of the TCP ACK otherwise and not in context of TP transmission request.



Note

Please consider `SoAdTcpTxQueueSize` since even if TP transmission is finished corresponding queue element is not released until reception of TCP ACK.

6.2.4.10 Trigger Transmit API for `SoAd_IfTransmit`

According to [1] the Trigger Transmit API is used in Socket Adaptor on transmission via `SoAd_IfRoutingGroupTransmit` or `SoAd_IfSpecificRoutingGroupTransmit`.

With this feature Trigger Transmit is also possible for transmissions via `SoAd_IfTransmit` like AUTOSAR specifies for other interface modules.



Caution

`SoAd_IfTransmit` has to be called with valid length (e.g. length of PDU instead of zero) since length is needed to call `Tcplp` transmission service before Trigger Transmit callback is called.

For some reasons length of PDU maybe unknown on call to `SoAd_IfTransmit`. In this case enable `SoAdTxDynamicLengthEnabled` in configuration tool to switch to a Vector specific transmission API between SoAd and `Tcplp`. With this adapted API it is possible to request a specific length but copy less data. So if PDU length is unknown maximum PDU length can be used in `SoAd_IfTransmit` and smaller length can be copied in Trigger Transmit callback.



Caution

The adapted API has to be supported by the TcpIp module. In case of BSD Socket API feature is always supported.

To enable this feature enable the corresponding parameter on the PduRoute (Figure 6-32).

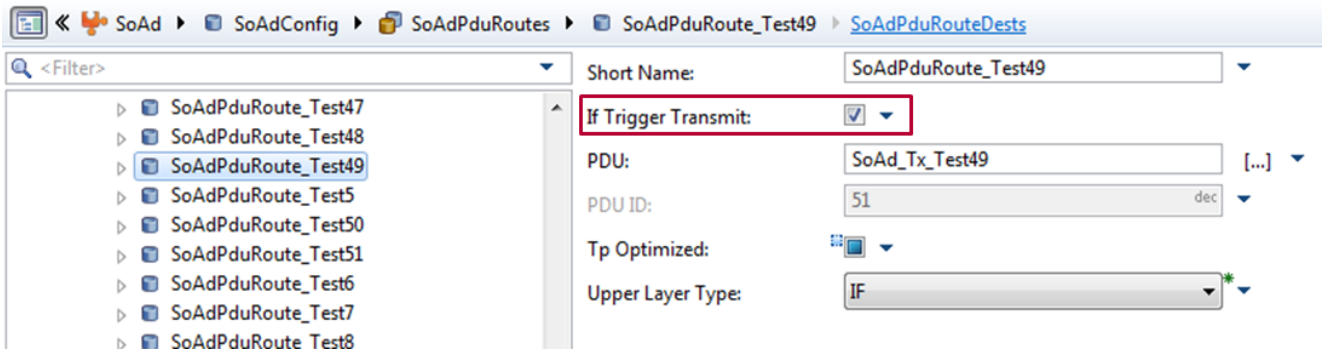


Figure 6-32 Trigger Transmit API for SoAd_IfTransmit configuration

6.2.4.11 Optimized buffer handling for PDU fan-out

If nPdu transmission is used on UDP sockets with IF-API a buffer is generated for each socket connection (nPduUdpTxBuffer). To save RAM a PDU can be configured to use the Trigger Transmit API (refer to 6.2.4.10). In combination with nPdu a transmission request to this PDU is stored in a queue (nPduUdpTxQueue) instead of entire PDU in a buffer and if trigger condition is fulfilled for the nPdu PDU data are retrieved via the Trigger Transmit API. The PDU will be stored once in the queue even if PDU transmission is triggered multiple times. Further transmission requests to this PDU will update the length information stored in the queue.

The queue size is configurable. If a new element exceeds queue size the nPdu is sent like specified for exceeding nPduUdpTxBuffer. So if queue is used parameters `SoAdSocketnPduUdpTxBufferMin` and `SoAdSocketnPduUdpTxQueueSize` have to be considered as trigger conditions (refer to Figure 6-33) but `SoAdSocketnPduUdpTxBufferMin` is a transmission trigger condition only and no buffer is generated.

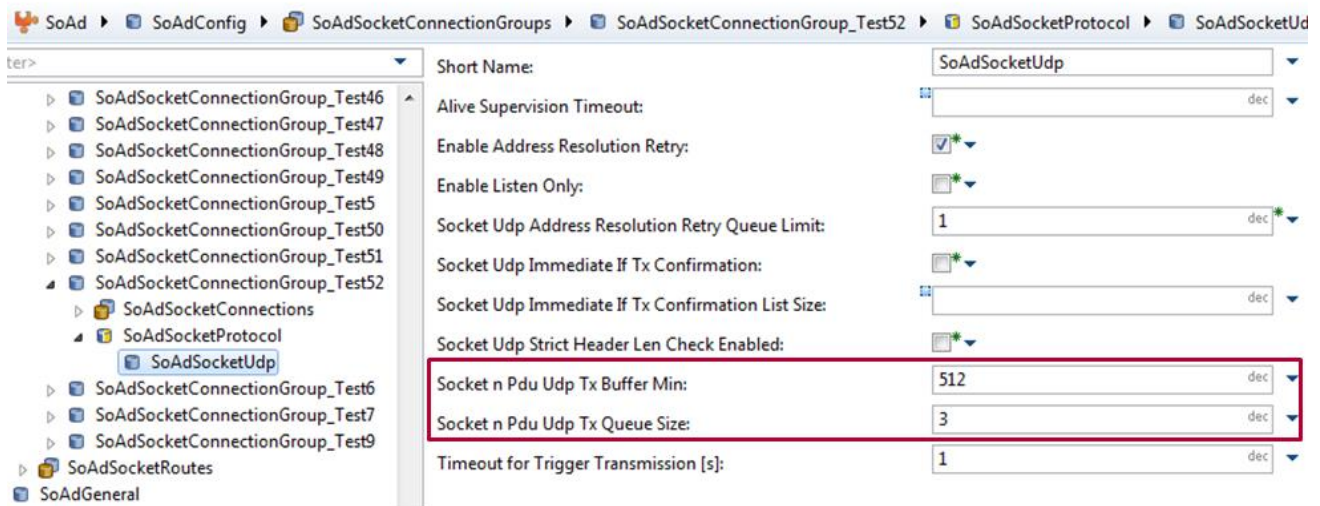


Figure 6-33 NPduUdpTxQueue configuration



Note

It is possible to have a nPduUdpTxBuffer and a nPduUdpTxQueue on one socket connection. In this case PDUs in nPduUdpTxBuffer are also represented by an element in the nPduUdpTxQueue.

6.2.4.12 Event Queues and Timeout Lists

6.2.4.12.1 Event Queues

Internally Socket Adaptor uses several event queues to store different events (e.g. socket connection states) to handle them in main function context.

It is possible to configure limitations for the event queues to reduce runtime in main function if many events occur at the same time. The limitation restricts the maximum number of executed events in one main function cycle. No events get lost if the limit is reached but they are executed in one of the next main function cycles (Figure 6-34).

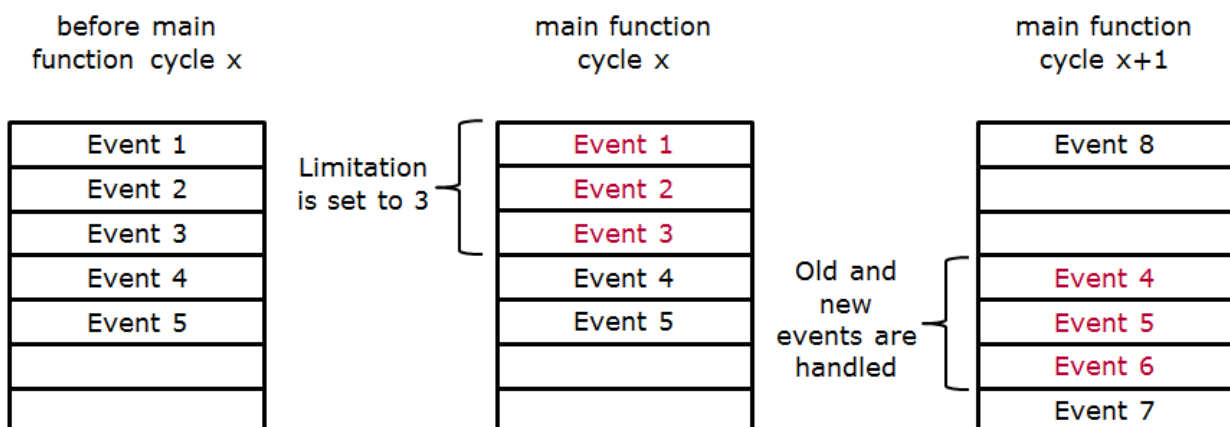


Figure 6-34 Event Queue limitation principle

To configure the event queue limitation create the corresponding configuration container and set the event queue limit (Figure 6-35). If container or parameter does not exist limitation is disabled (default) and all possible events are handled within one main function.

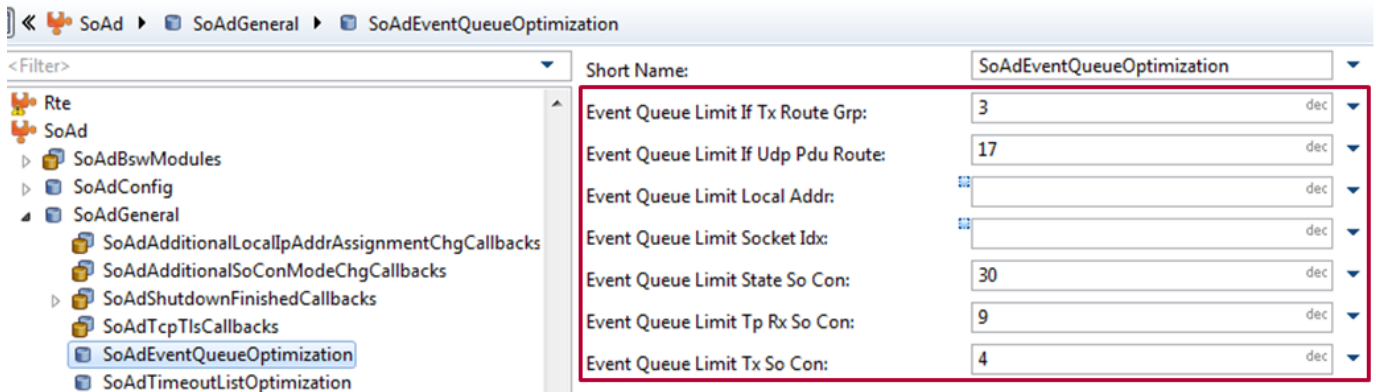


Figure 6-35 Event Queue limitation configuration

6.2.4.12.2 Timeout Lists

Beside event queues Socket Adaptor uses timeout lists to handle timeouts in main function (e.g. UDP alive supervision timeout).

It is also possible to configure limitations for the timeout lists. The limitation limits the size of the corresponding timeout list itself. So if limitation is reached a new timeout cannot be added to list and for example in case of Trigger Timeout for nPdu's the corresponding transmission request is rejected.

To configure the timeout list limitation create the corresponding configuration container and set the timeout list limit (Figure 6-36). If container or parameter does not exist limitation is disabled (default) and all possible timeouts can be handled.

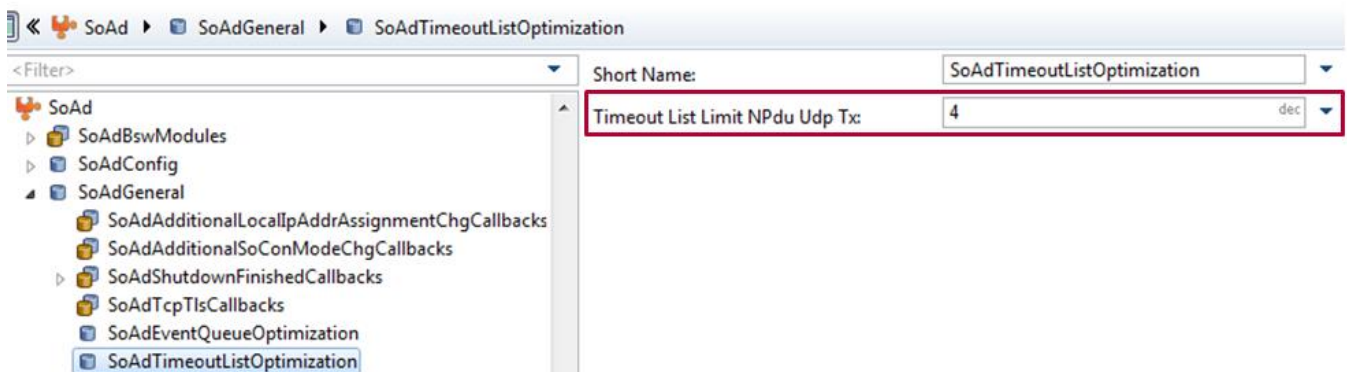


Figure 6-36 Timeout List limitation configuration

6.2.4.13 PDU reception verification

For TCP socket connections using the PDU Header option over the TP-API Socket Adaptor supports a PDU reception verification callback which can be used to filter a received PDU according to the following parameters:

1. Local IP address and port
2. Remote IP address and port

3. PDU Header ID
4. PDU data

This feature can be used to implement a firewall on Socket Adaptor level.

In case callback `<Up_VerifyRxPdu>` is successful Socket Adaptor forwards the PDU as configured. In case callback fails Socket Adaptor drops the PDU silently and continues with the reception of PDUs received afterwards.

Figure 6-37 shows how to configure the name of the callback and the maximum amount of PDU data which are forwarded via the callback.

Figure 6-38 shows how to enable the call of the callback for a specific socket connection. If disabled on a socket connection the callback won't be called.

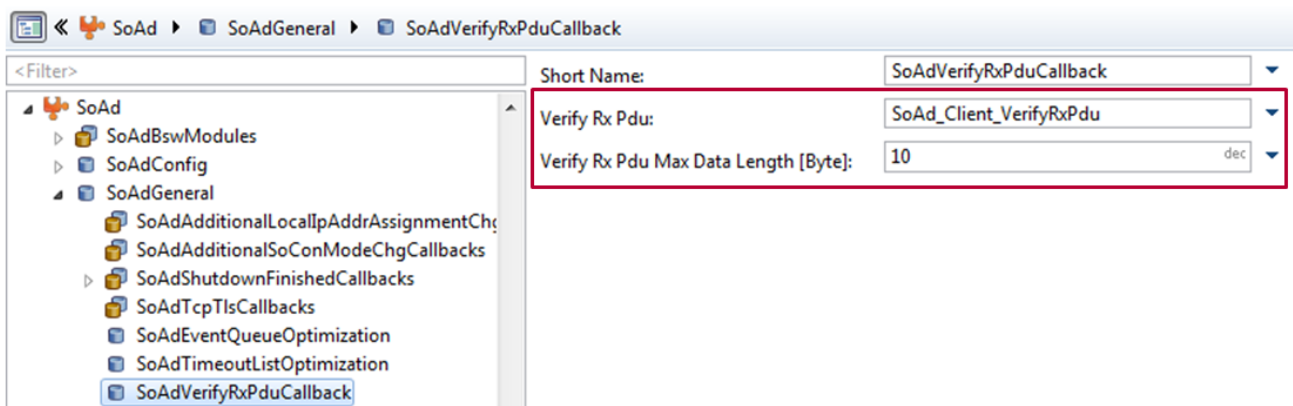


Figure 6-37 PDU reception verification callback configuration

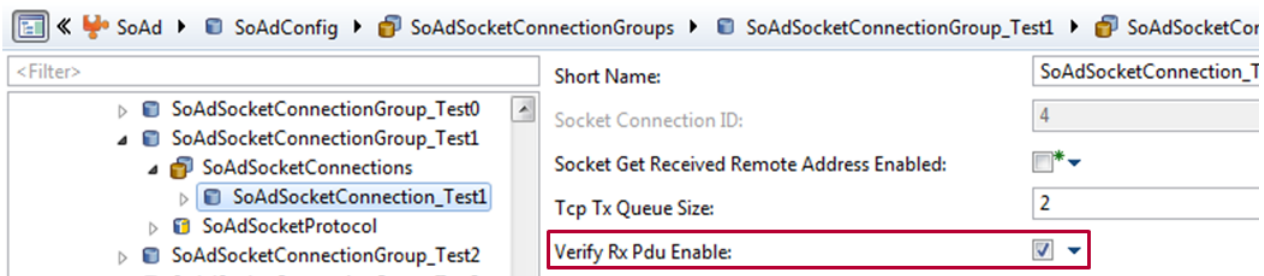


Figure 6-38 PDU reception verification socket connection configuration

6.2.4.14 “Transmission on specific socket connection” and “Forward socket connection on reception”

Both “Transmission on specific socket connection” and “Forward socket connection on reception” are features based on PDU meta data. These features are already described in a newer version of Socket Adaptor (see meta data requirements in [2]).

“Forward socket connection on reception” describes the mechanism to append the socket connection index as meta data to a received PDU. This feature can be used to get the socket connection index on which the PDU has been received.

“Transmission on specific socket connection” describes the mechanism to extract the socket connection index from meta data of a PDU which shall be transmitted. Socket Adaptor

transmits the PDU over the corresponding socket connection. This feature can be used to restrict the PDU transmission to a specific socket connection in case a PDU fan-out (see [1]) is configured.

Both features can be used to configure an optimization for the modeling of Client/Server Calls (i.e. methods). For more details please see “RfC 64808” in the AUTOSAR Bugzilla.

To enable the usage of the meta data create the following parameter for the corresponding PDU in the EcuC module:

`EcuC/EcucPduCollection/Pdu/MetaDataLength`

Set the value to 2 since 2 bytes is the size of the socket connection index.

6.2.4.15 Get DHCP option events

There are no indications as to when a DHCP option can be read from a received DHCP message or when a DHCP option can be written into a DHCP message. For this reason, callbacks can be configured for the user to be notified of DHCP message reception and DHCP message transmission events.

They can be configured using the parameter:

`SoAd/SoAdGeneral/SoAdDhcpEventCallback`

6.2.4.16 Keep Socket Connection online after transmission

According to AUTOSAR, a socket connection can be opened on UDP reception (SWS_SoAd_00592) or on TCP connection establishment (SWS_SoAd_00594). This applies if the configured remote address contains wildcards. After transmission of a PDU, the socket connection shall be closed (SWS_SoAd_00582 for UDP and SWS_SoAd_00644 for TCP).

In some applications it might make sense to keep a wildcard socket connection online after transmission to prevent a repeated opening and closing. This can be configured by enabling the corresponding parameter `SoAdSocketAutomaticSoConSetupKeepOnline` shown in Figure 6-39. The setting only applies if the automatic socket connection setup is enabled and if the socket connection group contains at least one remote address which is set to wildcard.

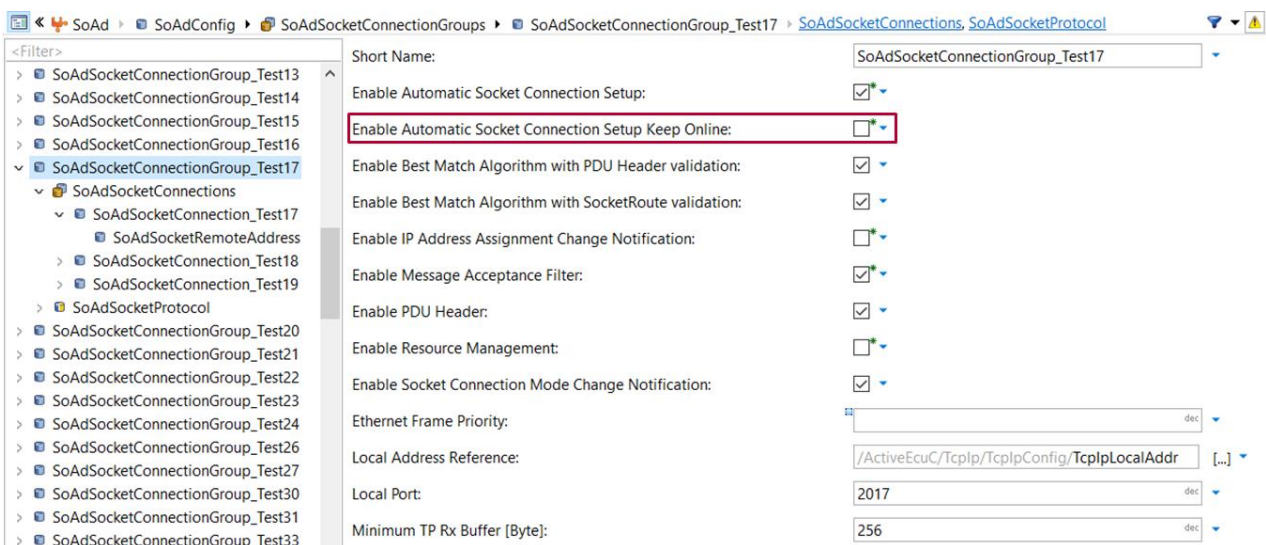


Figure 6-39 Keep Socket Connection online configuration

When this feature is enabled, it is recommended to use a timeout for automatic closing of the socket connections. For UDP, the `SoAdSocketUdpAliveSupervisionTimeout` can be used, which is shown in Figure 6-40. In case of a TCP socket connection it is required to enable `SoAdSocketTcpKeepAlive` and to set `SoAdSocketTcpKeepAliveProbesMax` to “0”, which is both configured in Figure 6-41. `SoAdSocketTcpKeepAliveTime` now defines the timeout value.

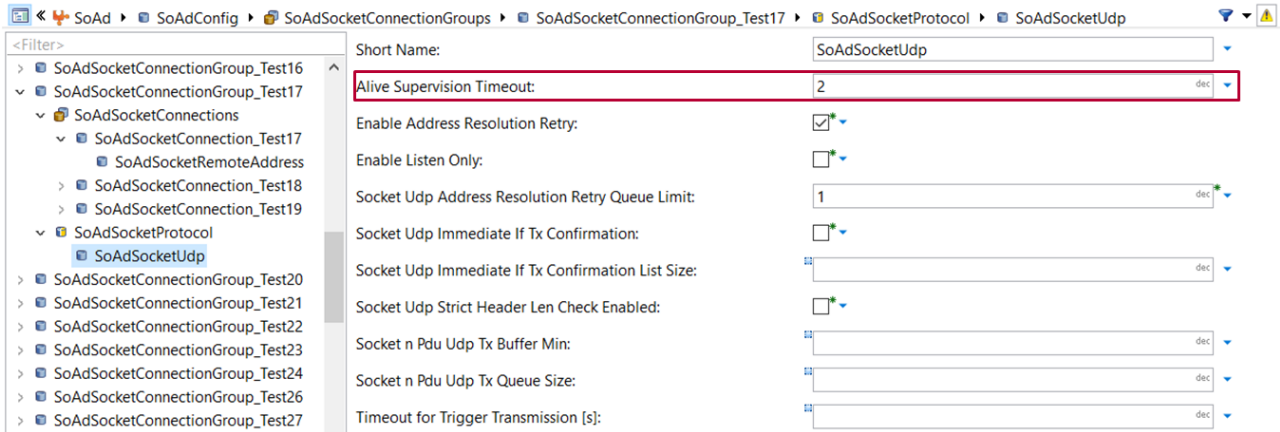


Figure 6-40 Socket UDP Alive Supervision Timeout configuration

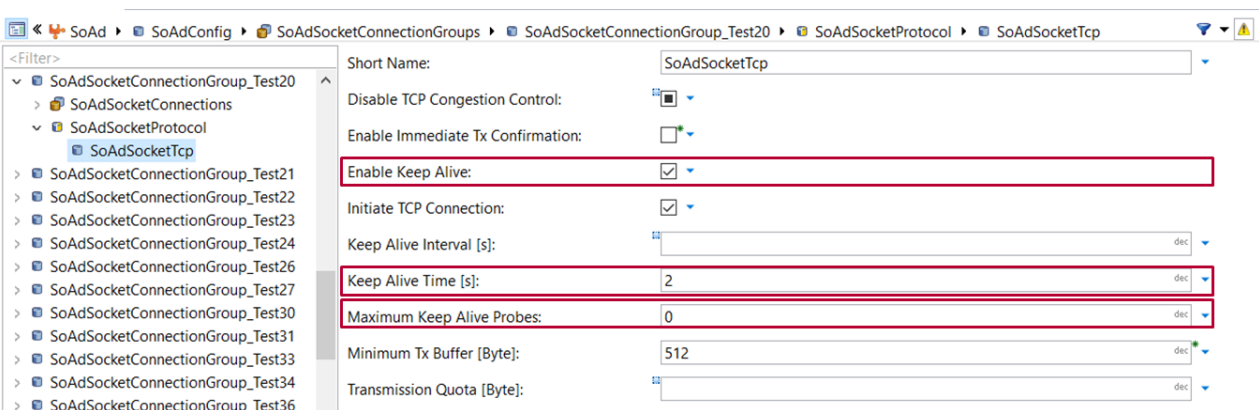


Figure 6-41 Socket TCP Keep Alive Time configuration

6.2.5 Complex Device Driver (CDD)

Chapter 3.2.5.1 describes which CDDs are supported by Socket Adaptor.

The expected API prefix of CDD within Socket Adaptor depends on the name of CDD. The name can be chosen on creation of a CDD within “Module Assistant” of DaVinci Configurator Pro (Figure 6-42).

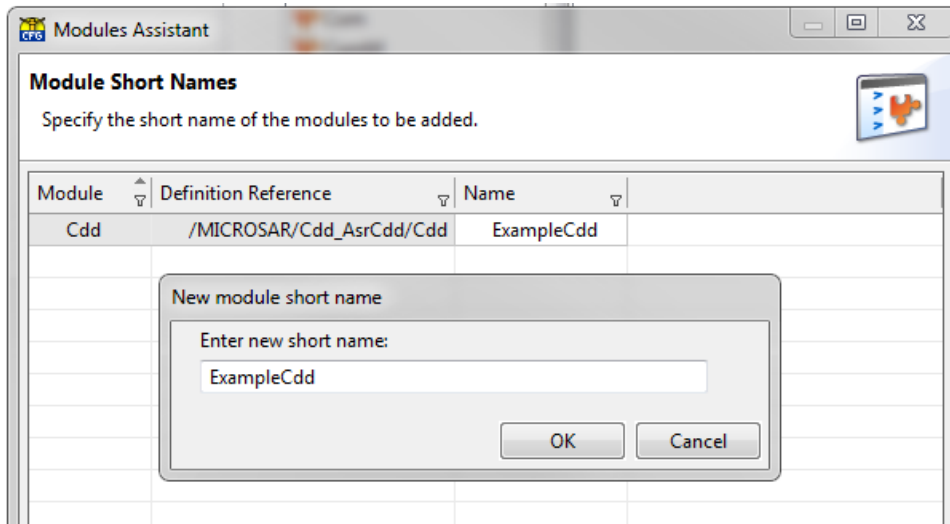


Figure 6-42 CDD configuration

For the example in Figure 6-42 the prefix is `ExampleCdd`. The prototype for an `IF RxIndication` would be `ExampleCdd_[SoAd][If]RxIndication().SoAd` and `If` infixes depends on configuration of “`ExampleCdd`” in “`SoAdBswModules`”.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
BSD	Berkeley sockets used by Linux for Ethernet communication
DaVinci Configurator Pro	Generation tool for MICROSAR components
GENy	Generation tool for CANbedded and MICROSAR components

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CDD	Complex Device Driver (Complex Driver)
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DoIP	Diagnostic over Internet Protocol
ECU	Electronic Control Unit
Eth	Ethernet Driver (MICROSAR)
EthTrcv	Ethernet Transceiver Driver (MICROSAR)
EthIf	Ethernet Interface (MICROSAR)
EthSM	Ethernet State Manager (MICROSAR)
IF	Interface API between BSW modules
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
PduR	PDU Router
RTE	Runtime Environment
SchM	Schedule Manager
SoAd	Short name of Socket Adaptor
SRS	Software Requirement Specification
SWS	Software Specification
Tcplp	Transport layer module containing TCP and UDP implementation (MICROSAR)
TCP	Transmission Control Protocol
Tls	Transport Layer Security module (MICROSAR)
TLS	Transport Layer Security
TP	Transport Protocol API between BSW modules
UDP	User Datagram Protocol

VLAN	Virtual Local Area Network
------	----------------------------

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com